

C# / .NET — Tập 2: Trung cấp

Generics · LINQ · async · xUnit · DI

2026-09-28

Mục lục

Tập 2 — C# Trung cấp

Chương 1 — Generics

Chương 2 — Delegates, lambda và events

Chương 3 — LINQ

Chương 4 — Records và pattern matching

Chương 5 — async / await

Chương 6 — Unit test với xUnit

Chương 7 — Dependency injection

GLOSSARY — Thuật ngữ (Tiếng Anh → Tiếng Việt)

Tập 2 — C# Trung cấp

Tập này dạy các công cụ bạn dùng mỗi ngày khi viết code C# "thật": generics, LINQ, async, test và dependency injection.

Mục lục

#	Chương	Ý chính	Ví dụ chạy được
1	Generics	generic class/method, constraints, variance	V2Ch01.Generics
2	Delegates, lambda và events	Func / Action, lambda, closure, event	V2Ch02.Delegates
3	LINQ	Where / Select / GroupBy / Join, deferred execution	V2Ch03.Linq
4	Records và pattern matching	record, with, switch expression, list/property patterns	V2Ch04.Records
5	async / await	Task, await, WhenAll, cancellation, IEnumerableAsync	V2Ch05.Async
6	Unit test với xUnit	[Fact], [Theory], fixture, dotnet test	V2Ch06.Calc, V2Ch06.Calc.Tests
7	Dependency injection	IServiceCollection, lifetime, options, Generic Host	V2Ch07.DI

Chạy ví dụ

```
cd books/csharp
./vol2-intermediate/run-examples.sh
```

Quy ước trình bày

- Thuật ngữ tiếng Anh giữ nguyên, giải thích tiếng Việt ở lần đầu; xem thêm Glossary cuối sách.
- Code và output trong sách được chép tự động từ file thật trong `examples/` sau mỗi lần chạy.
- Kiểm tra hiển thị tiếng Việt: ă ả ã ẽ ơ ư đ Ƴ Đ (nếu các chữ này hiện đúng, font đã đủ dấu).

Chương 1 — Generics

Mục tiêu

- Viết generic method và generic class.
- Dùng *constraint* (ràng buộc) `where T : ...`.
- Hiểu vì sao generics của C# khác Java (reified vs type erasure) và TypeScript.
- Biết *generic math* (`INumber<T>`) và *variance* (`out T`, `in T`) ở mức cơ bản.

Giải thích đơn giản

Generics = viết code một lần, dùng cho nhiều kiểu. Bạn đã dùng `List<T>` ở Tập 1: `List<int>`, `List<string>` là cùng một code, chỉ khác "tham số kiểu" `T`.

TypeScript có generics giống vậy: `function max<T>(a: T, b: T): T`. Điểm khác:

- TypeScript: kiểu biến mất sau khi biên dịch.
- Java: *type erasure* — lúc chạy `List<String>` chỉ là `List`.
- C#: *reified generics* — lúc chạy vẫn biết `List<int>` là `List<int>`. Nhờ vậy `new T()`, `typeof(T)`, `default(T)` đều dùng được, và `List<int>` không phải boxing.

Ví dụ

```
// 1. Generic method: một hàm dùng cho nhiều kiểu
static T Max<T>(T a, T b) where T : IComparable<T> => a.CompareTo(b) >= 0 ? a : b;

Console.WriteLine($"Max(3, 7) = {Max(3, 7)}");
Console.WriteLine($"Max(\"an\", \"binh\") = {Max(\"an\", \"binh\")}");

// 2. Generic class
var intStack = new SimpleStack<int>();
intStack.Push(1);
intStack.Push(2);
Console.WriteLine($"Pop = {intStack.Pop()}, Count = {intStack.Count}");

// 3. Generic giữ kiểu lúc chạy (reified), khác Java type erasure
Console.WriteLine($"typeof(List<int>) = {typeof(List<int>).Name}
<{typeof(List<int>).GenericTypeArguments[0].Name}>");
Console.WriteLine($"List<int> == List<string>? {typeof(List<int>) == typeof(List<string>)}");
Console.WriteLine($"C# 14 nameof(List<>) = {nameof(List<>) }");

// 4. Constraint: where T : class, new(), interface
var repo = new InMemoryRepository<Customer>();
repo.Add(new Customer { Id = 1, Name = "An" });
repo.Add(new Customer { Id = 2, Name = "Binh" });
Console.WriteLine($"Find(2) = {repo.Find(2)?.Name}; Find(9) = {repo.Find(9)?.Name ??
>null}");
Console.WriteLine($"CreateDefault: {InMemoryRepository<Customer>.CreateDefault().Name}");

// 5. Generic math (static abstract members trong interface, .NET 7+)
static T Sum<T>(IEnumerable<T> values) where T : System.Numerics.INumber<T>
{
    T total = T.Zero;
    foreach (var v in values) total += v;
    return total;
}
```

```

}
Console.WriteLine($"Sum<int> = {Sum([1, 2, 3])}, Sum<decimal> = {Sum([1.5m, 2.25m])}");

// 6. Variance: IEnumerable<out T> là covariant
IEnumerable<string> names = ["x", "y"];
IEnumerable<object> objects = names; // OK vì string là object
Console.WriteLine($"Covariance: {string.Join(", ", objects)}");

// 7. Result<T>: kiểu generic tự viết, hay dùng để trả lỗi không cần exception
Result<int> ok = Result<int>.Ok(42);
Result<int> fail = Result<int>.Fail("không tìm thấy");
Console.WriteLine($"{ok} | {fail}");

public class SimpleStack<T>
{
    private readonly List<T> _items = [];
    public int Count => _items.Count;
    public void Push(T item) => _items.Add(item);
    public T Pop()
    {
        if (_items.Count == 0) throw new InvalidOperationException("Stack rỗng");
        T last = _items[^1];
        _items.RemoveAt(_items.Count - 1);
        return last;
    }
}

public interface IEntity
{
    int Id { get; }
}

public class Customer : IEntity
{
    public int Id { get; init; }
    public string Name { get; init; } = "(mặc định)";
}

public class InMemoryRepository<T> where T : class, IEntity, new()
{
    private readonly Dictionary<int, T> _store = [];
    public void Add(T entity) => _store[entity.Id] = entity;
    public T? Find(int id) => _store.GetValueOrDefault(id);
    public static T CreateDefault() => new();
}

public readonly record struct Result<T>(bool IsSuccess, T? Value, string? Error)
{
    public static Result<T> Ok(T value) => new(true, value, null);
    public static Result<T> Fail(string error) => new(false, default, error);
    public override string ToString() => IsSuccess ? $"Ok({Value})" : $"Fail({Error})";
}

```

Output thật:

```

Max(3, 7) = 7
Max("an", "binh") = binh
Pop = 2, Count = 1
typeof(List<int>) = List`1<Int32>
List<int> == List<string>? False
C# 14 nameof(List<>) = List
Find(2) = Binh; Find(9) = null
CreateDefault: (mặc định)
Sum<int> = 6, Sum<decimal> = 3.75
Covariance: x,y
Ok(42) | Fail(không tìm thấy)

```

List1 là tên thật của kiểu List trong runtime (số 1 = có 1 tham số kiểu).

Đi sâu

Các loại constraint

Constraint	Ý nghĩa
where T : class	T là reference type
where T : struct	T là value type (không null)
where T : notnull	T không phải kiểu nullable
where T : new()	T có constructor không tham số
where T : BaseClass	T là BaseClass hoặc lớp con
where T : IInterface	T implement interface
where T : unmanaged	T là value type không chứa reference (dùng với Span, pointer)
where T : allows ref struct	T có thể là ref struct (C# 13)

Không có constraint, bạn chỉ gọi được các method của object trên T. Constraint Comparable<T> cho phép gọi a.CompareTo(b) trong Max<T>.

Generic math

Từ .NET 7, interface có thể có *static abstract member*. INumber<T> khai báo T.Zero, toán tử + ... nên Sum<T> ở trên chạy cho int, decimal, double mà không cần viết lại. Đây là tính năng Java và TypeScript chưa có.

Variance: out và in

- IEnumerable<out T> là *covariant*: IEnumerable<string> dùng được ở chỗ cần IEnumerable<object> (vì chỉ **đọc** T ra).
- Action<in T> là *contravariant*: Action<object> dùng được ở chỗ cần Action<string> (vì chỉ **nhận** T vào).
- List<T> **không** variant: List<string> không gán được cho List<object>, vì nếu được thì bạn có thể Add(123) vào list chuỗi.

Khi nào tự viết generic?

- Code giống hệt nhau, chỉ khác kiểu: repository, cache, Result<T>, Pair<T1, T2>.
- Đừng lạm dụng: nếu chỉ có một kiểu dùng, viết kiểu cụ thể dễ đọc hơn.

Lỗi và bẫy thường gặp

- Quên constraint** → lỗi "Operator '>' cannot be applied to operands of type 'T'". Thêm Comparable<T> hoặc INumber<T>.
- default(T) với reference type là null**: kiểm tra null khi trả về T?.
- Nghĩ List<Derived> là List<Base>**: không phải. Dùng IEnumerable<Base> nếu chỉ đọc.

- **Static field trong generic class:** mỗi `T` có một bản riêng (`Cache<int>` và `Cache<string>` không chung static field).

Tóm tắt

- Generics giúp viết code tái sử dụng, an toàn kiểu, không boxing.
- Constraint mở khóa các thao tác trên `T`.
- C# generics là reified: kiểu tồn tại lúc chạy.
- Generic math và variance là công cụ nâng cao nhưng rất hữu ích cho thư viện.

Bài tập (có lời giải)

1. Viết `record Pair<T1, T2>` có method `Swap()` trả về `Pair<T2, T1>`.
2. Viết `FindMax<T>(IEnumerable<T>)` với constraint phù hợp; ném exception nếu rỗng.
3. Viết `LruCache<TKey, TValue>` dung lượng cố định: khi đầy, loại phần tử lâu nhất chưa dùng.

▼ Lời giải

```
// Bài 1: Pair<T1, T2> với Swap
var pair = new Pair<string, int>("tuổi", 30);
Console.WriteLine($"Bài 1: {pair} -> {pair.Swap()}");

// Bài 2: FindMax<T> trên danh sách, ném lỗi nếu rỗng
Console.WriteLine($"Bài 2: {FindMax([3, 9, 2])}, {FindMax(["kiwi", "apple", "mango"])");
try { FindMax(Array.Empty<int>()); }
catch (InvalidOperationException ex) { Console.WriteLine($"Bài 2: {ex.Message}"); }

// Bài 3: LruCache<TKey, TValue> giới hạn dung lượng
var cache = new LruCache<string, int>(capacity: 2);
cache.Set("a", 1); cache.Set("b", 2);
_ = cache.TryGet("a", out _); // "a" vừa được dùng
cache.Set("c", 3);           // loại "b" (lâu nhất chưa dùng)
Console.WriteLine($"Bài 3: có a? {cache.TryGet("a", out _)}, có b? {cache.TryGet("b", out _)}, có c? {cache.TryGet("c", out _)}");

static T FindMax<T>(IEnumerable<T> items) where T : IComparable<T>
{
    if (items.Count == 0) throw new InvalidOperationException("Danh sách rỗng");
    T max = items[0];
    foreach (var item in items)
        if (item.CompareTo(max) > 0) max = item;
    return max;
}

public record Pair<T1, T2>(T1 First, T2 Second)
{
    public Pair<T2, T1> Swap() => new(Second, First);
}

public class LruCache<TKey, TValue>(int capacity) where TKey : notnull
{
    private readonly Dictionary<TKey, LinkedListNode<(TKey Key, TValue Value)>> _map = [];
    private readonly LinkedList<(TKey Key, TValue Value)> _order = new();

    public void Set(TKey key, TValue value)
    {
        if (_map.Remove(key, out var old)) _order.Remove(old);
        else if (_map.Count == capacity)
        {

```

```

        var lru = _order.Last!;
        _order.RemoveLast();
        _map.Remove(lru.Value.Key);
    }
    _map[key] = _order.AddFirst((key, value));
}

public bool TryGet(TKey key, out TValue? value)
{
    if (_map.TryGetValue(key, out var node))
    {
        _order.Remove(node);
        _order.AddFirst(node);
        value = node.Value.Value;
        return true;
    }
    value = default;
    return false;
}
}

```

Bài 1: Pair { First = tuổi, Second = 30 } -> Pair { First = 30, Second = tuổi }

Bài 2: 9, mango

Bài 2: Danh sách rỗng

Bài 3: có a? True, có b? False, có c? True

Bài 3 kết hợp Dictionary (tra cứu O(1)) và LinkedList (đổi thứ tự O(1)).

Nguồn tham khảo (Sources)

- Generic types (C# fundamentals): <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/types/generics.md>
- Generics in .NET: <https://github.com/dotnet/docs/blob/main/docs/standard/generics.md>
- Constraints on type parameters: <https://github.com/dotnet/docs/blob/main/docs/csharp/programming-guide/generics/constraints-on-type-parameters.md>
- Covariance and contravariance: <https://github.com/dotnet/docs/blob/main/docs/standard/generics/covariance-and-contravariance.md>
- Generic math: <https://github.com/dotnet/docs/blob/main/docs/standard/generics/math.md>
- Unbound generic types in nameof (C# 14): <https://github.com/dotnet/csharplang/blob/main/proposals/csharp-14.0/unbound-generic-types-in-nameof.md>

Chương 2 — Delegates, lambda và events

Mục tiêu

- Hiểu *delegate* (kiểu đại diện cho hàm) và dùng `Func`, `Action`, `Predicate`.
- Viết *lambda expression* và hiểu *closure*.
- Dùng *multicast delegate* và *event* theo mẫu publisher/subscriber.
- Biết tính năng C# 14: modifier (`out`, `ref`) trên tham số lambda không cần ghi kiểu.

Giải thích đơn giản

Trong TypeScript, hàm là giá trị: `const f = (x: number) => x * 2`; và kiểu của nó là `(x: number) => number`. Trong C#, **delegate** là "kiểu của hàm":

TypeScript	C#
<code>(x: number) => number</code>	<code>Func<int, int></code>
<code>(msg: string) => void</code>	<code>Action<string></code>
<code>(n: number) => boolean</code>	<code>Predicate<int></code> hoặc <code>Func<int, bool></code>
<code>() => void</code>	<code>Action</code>

Quy tắc nhớ: `Func<T1, T2, ..., TResult>` — **kiểu cuối cùng là kiểu trả về**. `Action<...>` không trả về gì (`void`).

Event là delegate được "bảo vệ": bên ngoài chỉ được `+=` (đăng ký) và `-=` (hủy), không được gọi hay gán đè. Giống `addEventListener` trong DOM.

Ví dụ

```
// 1. Delegate tự khai báo
Calculate add = (a, b) => a + b;
Calculate mul = Multiply; // method group
Console.WriteLine($"add(2,3)={add(2, 3)}, mul(2,3)={mul(2, 3)}");

// 2. Func và Action có sẵn
Func<int, int> square = x => x * x;
Func<string, int, string> repeat = (s, n) => string.Concat(Enumerable.Repeat(s, n));
Action<string> log = msg => Console.WriteLine($"[log] {msg}");
Predicate<int> isEven = n => n % 2 == 0;
log($"square(5)={square(5)}, repeat(\"ab\",3)={repeat(\"ab\", 3)}, isEven(4)={isEven(4)}");

// 3. Hàm nhận hàm (higher-order function)
static List<TOut> Map<TIn, TOut>(IEnumerable<TIn> source, Func<TIn, TOut> f)
{
    var result = new List<TOut>();
    foreach (var item in source) result.Add(f(item));
    return result;
}
Console.WriteLine($"Map: {string.Join(", ", Map([1, 2, 3], x => $"#{x}"))}");

// 4. Closure: lambda "nhớ" biến bên ngoài
Func<int> MakeCounter()
```

```

{
    int count = 0;
    return () => ++count;
}
var next = MakeCounter();
Console.WriteLine($"Counter: {next()}, {next()}, {next()}");

// 5. Multicast delegate: một delegate gọi nhiều hàm
Action<string> pipeline = s => Console.WriteLine($" bước 1: {s}");
pipeline += s => Console.WriteLine($" bước 2: {s.ToUpper()}");
pipeline("xin chào");

// 6. Event: publisher / subscriber
var thermostat = new Thermostat();
thermostat.TemperatureChanged += (sender, e) =>
    Console.WriteLine($" Nhiệt độ: {e.OldValue} -> {e.NewValue}");
EventHandler<TemperatureChangedEventArgs> alarm = (_, e) =>
{
    if (e.NewValue > 30) Console.WriteLine(" CẢNH BÁO: quá nóng!");
};
thermostat.TemperatureChanged += alarm;
thermostat.Temperature = 25;
thermostat.Temperature = 32;
thermostat.TemperatureChanged -= alarm; // hủy đăng ký
thermostat.Temperature = 35;
thermostat.Temperature = 35; // không đổi -> không phát event

// 7. C# 14: modifier trên tham số lambda không cần ghi kiểu
TryParser<int> parse = (text, out result) => int.TryParse(text, out result);
Console.WriteLine($"parse(\"12\") -> {parse(\"12\", out var n)} ({n})");

static int Multiply(int a, int b) => a * b;

delegate int Calculate(int a, int b);
delegate bool TryParser<T>(string text, out T result);

public class TemperatureChangedEventArgs : EventArgs
{
    public double OldValue { get; } = oldValue;
    public double NewValue { get; } = newValue;
}

public class Thermostat
{
    public event EventHandler<TemperatureChangedEventArgs>? TemperatureChanged;

    public double Temperature
    {
        get;
        set
        {
            if (field == value) return;
            var old = field;
            field = value;
            TemperatureChanged?.Invoke(this, new TemperatureChangedEventArgs(old, value));
        }
    }
}

```

Output thật:

```

add(2,3)=5, mul(2,3)=6
[log] square(5)=25, repeat("ab",3)=ababab, isEven(4)=True
Map: #1, #2, #3
Counter: 1, 2, 3
    bước 1: xin chào

```

```
bước 2: XIN CHÀO
Nhiệt độ: 0 -> 25
Nhiệt độ: 25 -> 32
CẢNH BÁO: quá nóng!
Nhiệt độ: 32 -> 35
parse("12") -> True (12)
```

Dòng `Nhiệt độ: 32 -> 35` không kèm cảnh báo, vì `alarm` đã bị hủy đăng ký (`--`). Lần gán `35` thứ hai không in gì vì setter bỏ qua giá trị không đổi.

Đi sâu

Delegate là object

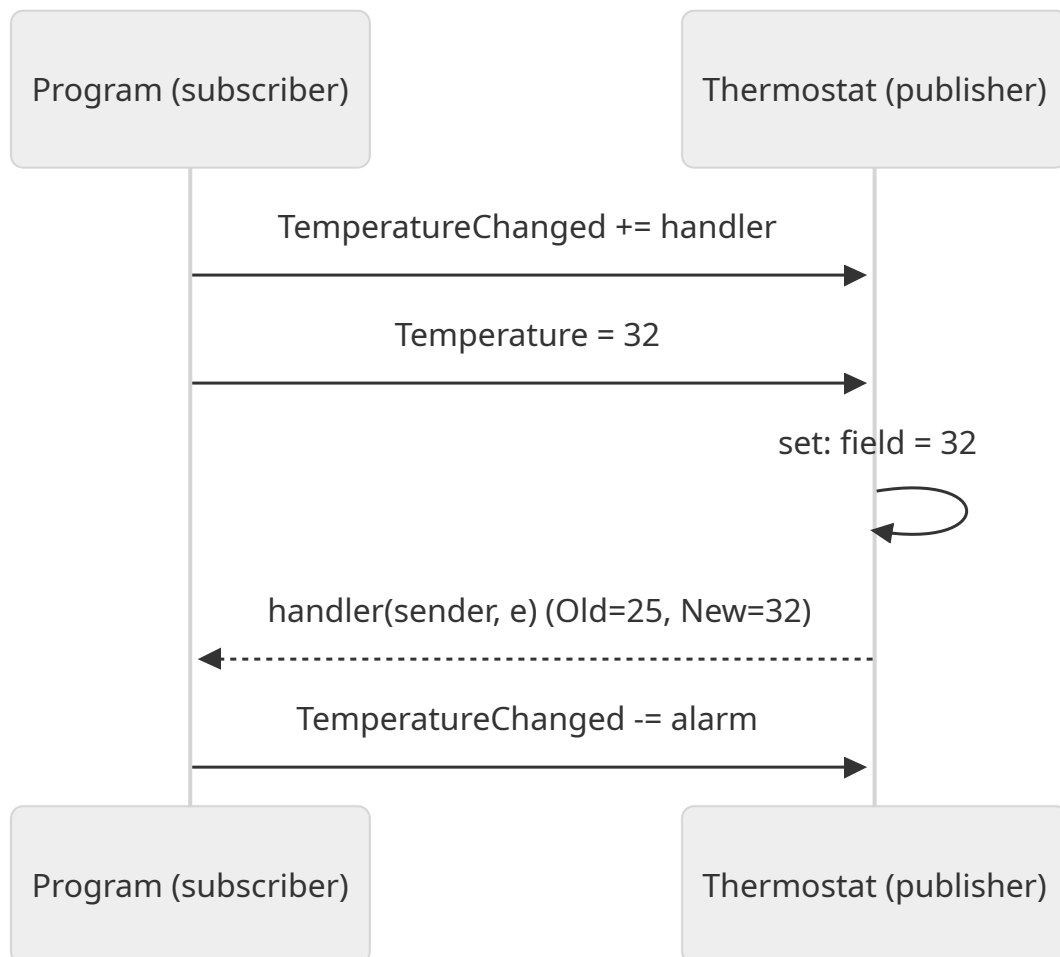
`Calculate add = (a, b) => a + b`; tạo một object delegate trỏ tới hàm. Có thể truyền nó làm tham số, lưu vào field, trả về từ hàm. LINQ (Chương 3) được xây trên `Func<...>`.

Closure

Lambda trong `MakeCounter` "bắt" (capture) biến `count`. Compiler chuyển `count` thành field của một class ẩn, nên biến sống lâu hơn hàm tạo ra nó. Giống closure trong JavaScript.

Bẫy kinh điển: capture biến vòng lặp `for` — mọi lambda thấy **cùng một** biến. `foreach` thì an toàn (mỗi vòng một biến mới, từ C# 5).

Mẫu event chuẩn của .NET



- Khai báo: `public event EventHandler<TArgs>? Name;`
- `TArgs` kế thừa `EventArgs`, chứa dữ liệu sự kiện.
- Phát sự kiện: `Name?.Invoke(this, args);` — `?.` để an toàn khi chưa ai đăng ký.

Lambda trong C# 14

Trước C# 14, nếu một tham số lambda có `out/ref` thì phải ghi kiểu cho **mọi** tham số: `(string text, out int result) => ...`. C# 14 cho phép `(text, out result) => ...` (xem `TryParser<int>` ở ví dụ).

Delegate hay interface?

- Chỉ một hành động, ngắn → `delegate/lambda` (`Func<T, bool>` để lọc).
- Nhiều hành động liên quan, có trạng thái, cần DI → `interface` (Chương 7).

Lỗi và bẫy thường gặp

- **Quên hủy đăng ký event:** publisher sống lâu giữ tham chiếu tới subscriber → rò rỉ bộ nhớ (memory leak). Luôn `-=` khi subscriber bị hủy.

- **Gọi event khi null:** `Name(this, e)` ném `NullReferenceException` nếu chưa ai đăng ký. Dùng `Name?.Invoke(...)`.
- **Exception trong một handler** của multicast delegate làm các handler sau không chạy.
- **async void lambda cho event:** exception không bắt được bằng `try/catch` bên ngoài. Chỉ dùng `async void` cho event handler và luôn tự `try/catch` bên trong.
- **Capture biến trong for** → mọi lambda dùng giá trị cuối cùng.

Tóm tắt

- Delegate = kiểu của hàm; thường dùng `Func<>/Action<>` có sẵn.
- Lambda `x => ...` tạo delegate; closure bắt biến bên ngoài.
- `event` = delegate chỉ cho `+=/-=` từ bên ngoài; theo mẫu `EventHandler<TArgs>`.

Bài tập (có lời giải)

1. Viết `Retry<T>(Func<T> action, int maxAttempts, Action<Exception>? onError)`.
2. Viết `Compose(f, g)` trả về hàm `x => g(f(x))`.
3. Viết class `Stock` có event `PriceChanged`, chỉ phát khi giá đổi hơn 5% so với lần báo trước.

▼ Lời giải

```
// Bài 1: Retry<T>(Func<T>, maxAttempts)
int attempts = 0;
int result = Retry(() =>
{
    attempts++;
    if (attempts < 3) throw new TimeoutException($"lần {attempts} lỗi");
    return 42;
}), maxAttempts: 5, onError: ex => Console.WriteLine($"Bài 1: thử lại vì: {ex.Message}"));
Console.WriteLine($"Bài 1: kết quả {result} sau {attempts} lần");

// Bài 2: Compose hai hàm
Func<int, int> addOne = x => x + 1;
Func<int, int> twice = x => x * 2;
var addThenTwice = Compose(addOne, twice);
Console.WriteLine($"Bài 2: Compose(addOne, twice)(5) = {addThenTwice(5)}");

// Bài 3: event PriceChanged trên Stock, chỉ báo khi thay đổi > 5%
var stock = new Stock("VNM", 100m);
stock.PriceChanged += (_, e) => Console.WriteLine($"Bài 3: {e.Symbol} {e.Old} -> {e.New} ({e.PercentChange:+0.0;-0.0}%)");
foreach (var p in new[] { 103m, 110m, 109m, 95m }) stock.Price = p;

static T Retry<T>(Func<T> action, int maxAttempts, Action<Exception>? onError = null)
{
    for (int i = 1; ; i++)
    {
        try { return action(); }
        catch (Exception ex) when (i < maxAttempts)
        {
            onError?.Invoke(ex);
        }
    }
}

static Func<T, TResult> Compose<T, TMid, TResult>(Func<T, TMid> first, Func<TMid, TResult>
second)
    => x => second(first(x));
```

```

public record PriceChangedEventArgs(string Symbol, decimal Old, decimal New)
{
    public decimal PercentChange => (New - Old) / Old * 100;
}

public class Stock(string symbol, decimal price)
{
    private decimal _lastNotified = price;
    public event EventHandler<PriceChangedEventArgs>? PriceChanged;

    public decimal Price
    {
        get;
        set
        {
            field = value;
            var args = new PriceChangedEventArgs(symbol, _lastNotified, value);
            if (Math.Abs(args.PercentChange) > 5)
            {
                _lastNotified = value;
                PriceChanged?.Invoke(this, args);
            }
        }
    } = price;
}

```

Bài 1: thử lại vì: lần 1 lỗi
 Bài 1: thử lại vì: lần 2 lỗi
 Bài 1: kết quả 42 sau 3 lần
 Bài 2: Compose(addOne, twice)(5) = 12
 Bài 3: VNM 100 -> 110 (+10.0%)
 Bài 3: VNM 110 -> 95 (-13.6%)

Bài 1 dùng *exception filter* `when (i < maxAttempts)`: lần cuối cùng exception không bị bắt và bay ra ngoài cho người gọi xử lý.

Nguồn tham khảo (Sources)

- Delegates and lambdas (C# fundamentals): <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/types/delegates-lambdas.md>
- Lambda expressions: <https://github.com/dotnet/docs/blob/main/docs/csharp/language-reference/operators/lambda-expressions.md>
- Events (C# programming guide): <https://github.com/dotnet/docs/blob/main/docs/csharp/programming-guide/events/index.md>
- Handling and raising events (.NET): <https://github.com/dotnet/docs/blob/main/docs/standard/events/index.md>
- Simple lambda parameters with modifiers (C# 14): <https://github.com/dotnet/csharplang/blob/main/proposals/csharp-14.0/simple-lambda-parameters-with-modifiers.md>

Chương 3 — LINQ

Mục tiêu

- Viết truy vấn LINQ bằng *method syntax* và *query syntax*.
- Dùng các toán tử chính: Where, Select, OrderBy, GroupBy, Join, Any, Sum, First, Skip/Take, ToDictionary, SelectMany, CountBy.
- Hiểu *deferred execution* (thực thi trì hoãn) và vì sao nó quan trọng.

Giải thích đơn giản

LINQ (Language Integrated Query) là bộ method để lọc, biến đổi, nhóm dữ liệu. Nếu bạn biết `array.filter().map()` trong TypeScript hoặc `stream().filter().map()` trong Java, bạn đã hiểu 80% LINQ:

TypeScript	Java Stream	LINQ
<code>filter</code>	<code>filter</code>	<code>Where</code>
<code>map</code>	<code>map</code>	<code>Select</code>
<code>flatMap</code>	<code>flatMap</code>	<code>SelectMany</code>
<code>sort</code>	<code>sorted</code>	<code>OrderBy / ThenBy</code>
<code>find</code>	<code>findFirst</code>	<code>First / FirstOrDefault</code>
<code>some / every</code>	<code>anyMatch / allMatch</code>	<code>Any / All</code>
<code>reduce</code>	<code>reduce</code>	<code>Aggregate</code>
<code>slice</code>	<code>skip / limit</code>	<code>Skip / Take</code>

LINQ chạy trên mọi `IEnumerable<T>` (List, mảng, Dictionary...). Với EF Core (Tập 3), cùng code LINQ được dịch sang SQL.

Ví dụ

```
List<Employee> employees =
[
    new("An", "Backend", 2500, 2019),
    new("Bình", "Frontend", 2200, 2021),
    new("Chi", "Backend", 3100, 2017),
    new("Dũng", "QA", 1800, 2022),
    new("Em", "Frontend", 2600, 2018),
];
List<Department> departments = [new("Backend", "Tầng 3"), new("Frontend", "Tầng 2"), new("QA", "Tầng 1")];

// 1. Where + Select + OrderBy (method syntax)
var seniors = employees
    .Where(e => e.JoinedYear <= 2019)
    .OrderByDescending(e => e.Salary)
    .Select(e => $"{e.Name} ({e.Salary})");
Console.WriteLine("Senior: " + string.Join(", ", seniors));

// 2. Query syntax (giống SQL) - cùng kết quả
```

```

var seniors2 = from e in employees
                where e.JoinedYear <= 2019
                orderby e.Salary descending
                select $"{e.Name} ({e.Salary})";
Console.WriteLine("Query syntax giống nhau? " + seniors.SequenceEqual(seniors2));

// 3. Aggregate: Count, Sum, Average, Max, MaxBy
Console.WriteLine($"Số NV={employees.Count}, tổng lương={employees.Sum(e => e.Salary)}, TB=
{employees.Average(e => e.Salary):F0}");
Console.WriteLine($"Lương cao nhất: {employees.MaxBy(e => e.Salary)!.Name}");
Console.WriteLine($"Có ai lương < 2000? {employees.Any(e => e.Salary < 2000)}; tất cả > 1000?
{employees.All(e => e.Salary > 1000)}");

// 4. GroupBy
foreach (var g in employees.GroupBy(e => e.Team).OrderBy(g => g.Key))
    Console.WriteLine($" {g.Key,-8}: {g.Count()} người, TB {g.Average(e => e.Salary):F0}");

// 5. Join
var withFloor = employees.Join(departments, e => e.Team, d => d.Name, (e, d) => $"
{e.Name}@{d.Floor}");
Console.WriteLine("Join: " + string.Join(", ", withFloor));

// 6. First / FirstOrDefault / Single
Console.WriteLine($"First QA: {employees.First(e => e.Team == "QA").Name}");
Console.WriteLine($"FirstOrDefault DevOps: {employees.FirstOrDefault(e => e.Team ==
"DevOps")?.Name ?? "null"}");

// 7. Phân trang: Skip + Take; Chunk
var page2 = employees.OrderBy(e => e.Name).Skip(2).Take(2).Select(e => e.Name);
Console.WriteLine("Trang 2 (size 2): " + string.Join(", ", page2));
Console.WriteLine("Chunk(2): " + string.Join(" | ", employees.Select(e =>
e.Name).Chunk(2).Select(c => string.Join(", ", c))));

// 8. ToDictionary, Distinct, SelectMany
var byName = employees.ToDictionary(e => e.Name);
Console.WriteLine($"byName["Chi"].Team = {byName["Chi"].Team}");
Console.WriteLine("Teams: " + string.Join(", ", employees.Select(e => e.Team).Distinct()));
string[][] tags = [{"c#", "sql"}, {"ts", "c#"}];
Console.WriteLine("SelectMany: " + string.Join(", ", tags.SelectMany(t => t).Distinct()));

// 9. Deferred execution: query chỉ chạy khi duyệt
int calls = 0;
var query = employees.Where(e => { calls++; return e.Salary > 2000; });
Console.WriteLine($"Sau khi tạo query: calls={calls}");
var list = query.ToList();
Console.WriteLine($"Sau ToList(): calls={calls}, kết quả={list.Count}");
_ = query.Count();
Console.WriteLine($"Duyệt lần 2: calls={calls} (chạy lại từ đầu!)");

// 10. CountBy / AggregateBy (.NET 9+)
foreach (var (team, count) in employees.CountBy(e => e.Team))
    Console.WriteLine($"{{team}}={{count}} ");
Console.WriteLine();

record Employee(string Name, string Team, decimal Salary, int JoinedYear);
record Department(string Name, string Floor);

```

Output thật:

```

Senior: Chi (3100), Em (2600), An (2500)
Query syntax giống nhau? True
Số NV=5, tổng lương=12200, TB=2440
Lương cao nhất: Chi
Có ai lương < 2000? True; tất cả > 1000? True
    Backend : 2 người, TB 2800
    Frontend: 2 người, TB 2400

```

```

QA      : 1 người, TB 1800
Join: An@Tầng 3, Bình@Tầng 2, Chi@Tầng 3, Dũng@Tầng 1, Em@Tầng 2
First QA: Dũng
FirstOrDefault DevOps: null
Trang 2 (size 2): Chi, Dũng
Chunk(2): An,Bình | Chi,Dũng | Em
byName["Chi"].Team = Backend
Teams: Backend, Frontend, QA
SelectMany: c#, sql, ts
Sau khi tạo query: calls=0
Sau ToList(): calls=5, kết quả=4
Duyệt lần 2: calls=10 (chạy lại từ đầu!)
Backend=2 Frontend=2 QA=1

```

Đi sâu

Deferred execution

Hầu hết toán tử LINQ (Where, Select, OrderBy...) **không chạy ngay**. Chúng chỉ tạo một "công thức". Công thức chạy khi bạn:

- duyệt bằng `foreach`, hoặc
- gọi toán tử "kết thúc" (`ToList`, `ToArray`, `Count`, `Sum`, `First`, `Any`...).

Output ở mục 9 chứng minh: tạo query xong `calls=0`; `ToList()` chạy 5 lần; gọi `Count()` thêm lần nữa chạy lại 5 lần (`calls=10`). Nếu nguồn dữ liệu đắt (đọc DB, file), hãy `ToList()` một lần rồi dùng lại list.



First vs FirstOrDefault vs Single

Method	Không có phần tử	Nhiều phần tử
First	ném <code>InvalidOperationException</code>	lấy phần tử đầu
FirstOrDefault	trả default (null với class)	lấy phần tử đầu
Single	ném exception	ném exception
SingleOrDefault	trả default	ném exception

Dùng `Single` khi logic đảm bảo chỉ có đúng một (ví dụ tìm theo khóa chính).

Query syntax

`from e in employees where ... orderby ... select ...` được compiler dịch sang method syntax. Hai cách cho cùng kết quả (dòng Query syntax giống nhau? `True`). Method syntax phổ biến hơn; query syntax dễ đọc hơn khi có `join` hoặc `let` phức tạp.

Toán tử mới

- `MaxBy` / `MinBy` (.NET 6): lấy **phần tử** có giá trị lớn nhất, không chỉ giá trị.
- `Chunk` (.NET 6): chia thành các nhóm kích thước cố định.

- `CountBy` / `AggregateBy` (.NET 9): đếm/gộp theo khóa mà không cần `GroupBy`.

Lỗi và bẫy thường gặp

- **Duyệt query nhiều lần** → chạy lại nhiều lần (có thể gọi DB nhiều lần). `ToList()` một lần.
- **Side effect trong lambda** (như `calls++` ở ví dụ): chỉ dùng để minh họa; code thật nên giữ lambda "thuần" (pure).
- **First trên collection rỗng** → exception. Dùng `FirstOrDefault` và kiểm tra null.
- **Count() > 0 để kiểm tra có phần tử**: dùng `Any()` — dừng ngay khi thấy phần tử đầu.
- **Sửa collection nguồn khi đang duyệt query** → `InvalidOperationException`.
- **OrderBy(...).OrderBy(...)**: lần sắp xếp thứ hai xóa lần đầu. Dùng `ThenBy`.

Tóm tắt

- LINQ = filter/map/reduce của C#, chạy trên mọi `IEnumerable<T>`.
- Hầu hết toán tử là lười (deferred); `ToList`, `Count`, `First`... mới chạy query.
- Chọn đúng `First`/`FirstOrDefault`/`Single` theo ý nghĩa nghiệp vụ.

Bài tập (có lời giải)

1. Lấy 3 sản phẩm đắt nhất còn hàng.
2. Nhóm theo danh mục: số sản phẩm và tổng giá trị tồn kho (`Price * Stock`), sắp giảm dần.
3. Tìm 2 từ xuất hiện nhiều nhất trong một câu (không phân biệt hoa thường).
4. In tên các sản phẩm hết hàng, hoặc "không có" nếu không có sản phẩm nào.

▼ Lời giải

```
Product[] products =
[
    new("Laptop", "Máy tính", 25_000_000m, 5),
    new("Chuột", "Phụ kiện", 300_000m, 50),
    new("Bàn phím", "Phụ kiện", 900_000m, 0),
    new("Màn hình", "Máy tính", 5_000_000m, 12),
    new("Tai nghe", "Phụ kiện", 1_200_000m, 8),
];

// Bài 1: 3 sản phẩm đắt nhất còn hàng
var top3 = products.Where(p => p.Stock > 0).OrderByDescending(p =>
p.Price).Take(3).Select(p => p.Name);
Console.WriteLine($"Bài 1: {string.Join(", ", top3)}");

// Bài 2: theo danh mục: số sản phẩm và tổng giá trị tồn kho
var byCategory = products
    .GroupBy(p => p.Category)
    .Select(g => new { Category = g.Key, Count = g.Count(), Value = g.Sum(p => p.Price *
p.Stock) })
    .OrderByDescending(x => x.Value);
foreach (var c in byCategory) Console.WriteLine($"Bài 2: {c.Category,-9} {c.Count} sp, tồn
kho {c.Value,15:N0}");

// Bài 3: 2 từ xuất hiện nhiều nhất (không phân biệt hoa thường)
string text = "C# và LINQ. LINQ rất mạnh; C# rất gọn. linq!";
var topWords = text
    .Split([' ', '.', ';', '!'], StringSplitOptions.RemoveEmptyEntries)
```

```

        .GroupBy(w => w.ToLowerInvariant())
        .OrderByDescending(g => g.Count()).ThenBy(g => g.Key)
        .Take(2)
        .Select(g => $"{g.Key}={g.Count()}");
Console.WriteLine($"Bài 3: {string.Join(", ", topWords)}");

// Bài 4: tên sản phẩm hết hàng (hoặc "không có")
var outOfStock = products.Where(p => p.Stock == 0).Select(p =>
p.Name).DefaultIfEmpty("không có");
Console.WriteLine($"Bài 4: hết hàng: {string.Join(", ", outOfStock)}");

record Product(string Name, string Category, decimal Price, int Stock);

```

```

Bài 1: Laptop, Màn hình, Tai nghe
Bài 2: Máy tính 2 sp, tồn kho 185,000,000
      Phụ kiện 3 sp, tồn kho 24,600,000
Bài 3: linq=3, c#=2
Bài 4: hết hàng: Bàn phím

```

Bài 2 dùng *anonymous type* `new { Category, Count, Value }` — tiện cho kết quả tạm thời trong một method. Bài 4 dùng `DefaultIfEmpty`.

Nguồn tham khảo (Sources)

- LINQ overview (C#): <https://github.com/dotnet/docs/blob/main/docs/csharp/linq/index.md>
- Introduction to LINQ queries: <https://github.com/dotnet/docs/blob/main/docs/csharp/linq/get-started/introduction-to-linq-queries.md>
- LINQ in .NET: <https://github.com/dotnet/docs/blob/main/docs/standard/linq/index.md>
- .NET 9 runtime libraries (CountBy, AggregateBy): <https://github.com/dotnet/docs/blob/main/docs/core/whats-new/dotnet-9/libraries.md>

Chương 4 — Records và pattern matching

Mục tiêu

- Dùng `record` và `record struct` cho dữ liệu bất biến, so sánh theo giá trị.
- Tạo bản sao bằng `with`, tách dữ liệu bằng *deconstruction*.
- Dùng pattern matching: type, property, positional, relational, logical, list, tuple pattern.
- Mô hình hóa dữ liệu kiểu "một trong nhiều loại" bằng record kế thừa + switch expression.

Giải thích đơn giản

Record là class (hoặc struct) mà compiler viết sẵn cho bạn:

- constructor, property `init` từ danh sách tham số,
- `Equals/==` so sánh **theo giá trị** (không theo địa chỉ),
- `ToString()` đẹp: `Person { Name = An, Age = 28 }`,
- `Deconstruct` và biểu thức `with` để copy có sửa.

Với TypeScript, nó giống `type Person = { readonly name: string; readonly age: number }` cộng thêm so sánh theo giá trị. Với Java, nó giống `record Person(String name, int age)`, cộng thêm `with`.

Pattern matching (so khớp mẫu) là cách hỏi "giá trị này có hình dạng thế nào?" và lấy dữ liệu ra cùng lúc. Nó giống *discriminated union* + `switch (x.kind)` trong TypeScript, nhưng mạnh hơn.

Ví dụ

```
// 1. Record: so sánh theo giá trị, ToString đẹp, with-expression
var p1 = new Person("An", 28);
var p2 = new Person("An", 28);
var p3 = p1 with { Age = 29 };
Console.WriteLine(p1);
Console.WriteLine($"p1 == p2: {p1 == p2}; ReferenceEquals: {ReferenceEquals(p1, p2)}; p3 = {p3}");

// Deconstruct
var (name, age) = p3;
Console.WriteLine($"Deconstruct: {name}, {age}");

// 2. record struct
var a = new Point(1, 2);
var b = a with { X = 5 };
Console.WriteLine($"{a} -> {b}");

// 3. Type pattern + property pattern + switch expression
Shape[] shapes = [new Circle(1), new Rectangle(2, 3), new Rectangle(4, 4), new Triangle(3, 4)];
foreach (var s in shapes)
{
    string desc = s switch
    {
        Circle { Radius: var r } => $"Hình tròn r={r}, S={Math.PI * r * r:F2}",
        Rectangle { Width: var w, Height: var h } when w == h => $"Hình vuông cạnh {w}",
        Rectangle(var w, var h) => $"Chữ nhật {w}x{h}", // positional pattern
        _ => $"Hình khác: {s.GetType().Name}",
    };
}
```

```

    };
    Console.WriteLine(" " + desc);
}

// 4. is + declaration pattern
object value = 42;
if (value is int n && n > 40) Console.WriteLine($"value là int lớn hơn 40: {n}");

// 5. Relational + logical patterns
static string Classify(int t) => t switch
{
    < 0 => "đóng băng",
    >= 0 and < 20 => "lạnh",
    >= 20 and < 30 => "dễ chịu",
    _ => "nóng",
};
Console.WriteLine($"Classify: {Classify(-5)}, {Classify(15)}, {Classify(25)}, {Classify(35)}");

// 6. List patterns
static string Describe(int[] xs) => xs switch
{
    [] => "rỗng",
    [var one] => $"một phần tử: {one}",
    [var first, .., var last] when first == last => $"đầu = cuối = {first}",
    [var first, .. var middle, var last] => $"đầu={first}, giữa có {middle.Length}, cuối={last}",
};
Console.WriteLine($"{{Describe([])}} | {{Describe([7])}} | {{Describe([1, 2, 1])}} | {{Describe([1, 2, 3, 4])}}");

// 7. Tuple pattern: máy trạng thái đơn giản
static string Next(string state, string action) => (state, action) switch
{
    ("Draft", "submit") => "Review",
    ("Review", "approve") => "Published",
    ("Review", "reject") => "Draft",
    (_, _) => state,
};
Console.WriteLine($"Draft -submit-> {Next("Draft", "submit")}; Review -reject-> {Next("Review", "reject")}; Published -submit-> {Next("Published", "submit")}");

// 8. Record kế thừa và so sánh theo kiểu
Order o1 = new OnlineOrder(1, 100m, "an@example.com");
Order o2 = new Order(1, 100m);
Console.WriteLine($"OnlineOrder == Order cùng dữ liệu? {o1 == o2}");

public record Person(string Name, int Age);
public readonly record struct Point(int X, int Y);

public abstract record Shape;
public record Circle(double Radius) : Shape;
public record Rectangle(double Width, double Height) : Shape;
public record Triangle(double A, double B) : Shape;

public record Order(int Id, decimal Total);
public record OnlineOrder(int Id, decimal Total, string Email) : Order(Id, Total);

```

Output thật:

```

Person { Name = An, Age = 28 }
p1 == p2: True; ReferenceEquals: False; p3 = Person { Name = An, Age = 29 }
Deconstruct: An, 29
Point { X = 1, Y = 2 } -> Point { X = 5, Y = 2 }
    Hình tròn r=1, S=3.14
    Chữ nhật 2x3

```

```
Hình vuông cạnh 4
Hình khác: Triangle
value là int lớn hơn 40: 42
Classify: đóng băng, lạnh, dễ chịu, nóng
rỗng | một phần tử: 7 | đầu = cuối = 1 | đầu=1, giữa có 2, cuối=4
Draft -submit-> Review; Review -reject-> Draft; Published -submit-> Published
OnlineOrder == Order cùng dữ liệu? False
```

Đi sâu

Các loại pattern

Pattern	Ví dụ	Ý nghĩa
Type / declaration	<code>x is int n</code>	x là int, gán vào n
Constant	<code>x is null, "Sat"</code>	bằng hằng số
Relational	<code>< 0, >= 20</code>	so sánh
Logical	<code>>= 0 and < 20, not null, A or B</code>	kết hợp
Property	<code>Circle { Radius: var r }</code>	kiểm tra/lấy property
Positional	<code>Rectangle(var w, var h)</code>	dùng Deconstruct
List	<code>[var first, .., var last]</code>	khớp mảng/list theo vị trí
Tuple	<code>(state, action) switch { ("Draft", "submit") => ... }</code>	khớp nhiều giá trị
Discard	<code>_</code>	mọi giá trị

Thứ tự nhánh quan trọng

Trong ví dụ, nhánh "Hình vuông" (có `when w == h`) phải đứng **trước** nhánh `Rectangle(var w, var h)`. Nếu đảo lại, nhánh tổng quát sẽ bắt hết. Compiler báo lỗi nếu một nhánh không bao giờ tới được (CS8510).

Record kế thừa và so sánh

`OnlineOrder == Order` trả về `False` dù `Id` và `Total` giống nhau: record so sánh cả **kiểu thực tế** (qua property ẩn `EqualityContract`). Điều này tránh lỗi coi hai loại khác nhau là bằng nhau.

record class VS record struct VS class

	record (class)	readonly record struct	class
Loại	reference	value	reference
<code>==</code>	theo giá trị	theo giá trị	theo tham chiếu
<code>with</code>	có	có	không
Dùng cho	DTO, event, kết quả query, value object	dữ liệu rất nhỏ (Point, Money nhỏ)	entity có hành vi và định danh

Trong Web API (Tập 3), record rất hợp cho **DTO** (Data Transfer Object — object chở dữ liệu request/response).

Mô hình "một trong nhiều loại"

`abstract record Shape` + các `record` con + `switch expression` = cách phổ biến hiện nay để làm *discriminated union*. C# 15 (cùng .NET 11, đang preview) bổ sung `union` và *closed hierarchies* để compiler kiểm tra đủ mọi trường hợp — **UNVERIFIED** cú pháp cuối cùng khi phát hành chính thức.

Lỗi và bẫy thường gặp

- **Record chứa collection** (`List<T>`): `==` so sánh tham chiếu của list, không so sánh từng phần tử. `with` copy nông (shallow copy) — hai record dùng chung một list.
- **Dùng record cho EF Core entity**: entity cần định danh và thường bị sửa; so sánh theo giá trị gây khó hiểu. Dùng `class` cho entity, record cho DTO.
- **Quên _ trong switch expression**: compiler cảnh báo (thành lỗi khi `TreatWarningsAsErrors`), lúc chạy có thể ném `SwitchExpressionException`.
- **Đặt nhánh tổng quát trước nhánh cụ thể** → lỗi CS8510 hoặc logic sai với `when`.

Tóm tắt

- `record` = dữ liệu bất biến, so sánh theo giá trị, `with`, `ToString` đẹp.
- Pattern matching giúp code phân nhánh ngắn, an toàn, dễ đọc.
- Abstract record + switch expression là cách mô hình hóa "một trong nhiều loại".

Bài tập (có lời giải)

1. Viết record `Money(decimal Amount, string Currency)` với toán tử `+`; ném exception nếu khác loại tiền.
2. Tính phí thanh toán: thẻ VISA 2%, thẻ khác 3%, chuyển khoản > 2 triệu miễn phí (còn lại 5.000), tiền mặt 0 — dùng switch expression với property pattern.
3. Phân tích lệnh `"add milk 2"`, `"remove milk"`, `"list"` ... bằng list pattern.

▼ Lời giải

```
// Bài 1: record Money với phép cộng, kiểm tra cùng loại tiền
var a = new Money(100m, "VND");
var b = a with { Amount = 50m };
Console.WriteLine($"Bài 1: {a} + {b} = {a + b}; a == new Money(100, VND)? {a == new
Money(100m, "VND")}");
try { _ = a + new Money(1m, "USD"); }
catch (InvalidOperationException ex) { Console.WriteLine($"Bài 1: {ex.Message}"); }

// Bài 2: tính phí thanh toán bằng switch expression + property pattern
Payment[] payments = [new Card(1_000_000m, "VISA"), new Card(1_000_000m, "JCB"), new
BankTransfer(5_000_000m), new Cash(200_000m)];
foreach (var p in payments) Console.WriteLine($"Bài 2: {p} -> phí {Fee(p):N0}");

// Bài 3: phân tích lệnh bằng list pattern
foreach (var cmd in new[] { "add milk 2", "remove milk", "list", "add", "clear all now" })
    Console.WriteLine($"Bài 3: '{cmd}' -> {Parse(cmd.Split(' '))}");

static decimal Fee(Payment p) => p switch
{
    Card { Brand: "VISA" } c => c.Amount * 0.02m,
    Card c => c.Amount * 0.03m,
```

```

BankTransfer { Amount: > 2_000_000m } => 0m,
BankTransfer => 5_000m,
Cash => 0m,
_ => throw new ArgumentOutOfRangeException(nameof(p)),
};

static string Parse(string[] parts) => parts switch
{
    ["add", var item, var qty] when int.TryParse(qty, out var n) => $"Thêm {n} {item}",
    ["add", var item] => $"Thêm 1 {item}",
    ["remove", var item] => $"Xóa {item}",
    ["list"] => "Liệt kê",
    [var verb, ..] => $"Không hiểu lệnh '{verb}' với {parts.Length - 1} tham số",
    [] => "Lệnh rỗng",
};

public record Money(decimal Amount, string Currency)
{
    public static Money operator +(Money x, Money y) =>
        x.Currency == y.Currency
        ? x with { Amount = x.Amount + y.Amount }
        : throw new InvalidOperationException($"Không cộng được {x.Currency} với
{y.Currency}");
    public override string ToString() => $"{Amount:N0} {Currency}";
}

public abstract record Payment(decimal Amount);
public record Card(decimal Amount, string Brand) : Payment(Amount);
public record BankTransfer(decimal Amount) : Payment(Amount);
public record Cash(decimal Amount) : Payment(Amount);

```

Bài 1: 100 VND + 50 VND = 150 VND; a == new Money(100, VND)? True
 Bài 1: Không cộng được VND với USD
 Bài 2: Card { Amount = 1000000, Brand = VISA } -> phí 20,000
 Bài 2: Card { Amount = 1000000, Brand = JCB } -> phí 30,000
 Bài 2: BankTransfer { Amount = 5000000 } -> phí 0
 Bài 2: Cash { Amount = 200000 } -> phí 0
 Bài 3: 'add milk 2' -> Thêm 2 milk
 Bài 3: 'remove milk' -> Xóa milk
 Bài 3: 'list' -> Liệt kê
 Bài 3: 'add' -> Không hiểu lệnh 'add' với 0 tham số
 Bài 3: 'clear all now' -> Không hiểu lệnh 'clear' với 2 tham số

Bài 3: nhánh ["add", var item, var qty] when int.TryParse(...) vừa khớp vị trí vừa kiểm tra số lượng hợp lệ.

Nguồn tham khảo (Sources)

- Records (C# fundamentals): <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/types/records.md>
- Record (language reference): <https://github.com/dotnet/docs/blob/main/docs/csharp/language-reference/builtin-types/record.md>
- Pattern matching overview: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/patterns/pattern-matching.md>
- Patterns (language reference): <https://github.com/dotnet/docs/blob/main/docs/csharp/language-reference/operators/patterns.md>
- List patterns: <https://github.com/dotnet/docs/blob/main/docs/csharp/fundamentals/patterns/list-patterns.md>

- C# 15 proposals (unions, closed hierarchies): <https://github.com/dotnet/csharplang/blob/main/Language-Version-History.md>

Chương 5 — async / await

Mục tiêu

- Viết hàm bất đồng bộ với `async Task / async Task<T>` và `await`.
- Chạy nhiều tác vụ song song bằng `Task.WhenAll`, `Task.WhenAny`.
- Hủy tác vụ bằng `CancellationToken` (kể cả timeout).
- Stream dữ liệu với `IAsyncEnumerable<T>` và `await foreach`.
- Biết khi nào dùng `ValueTask<T>`.

Giải thích đơn giản

Bạn đã quen `async/await` trong TypeScript. C# (có trước TypeScript) dùng cùng ý tưởng:

TypeScript	C#
<code>Promise<T></code>	<code>Task<T></code>
<code>Promise<void></code>	<code>Task</code>
<code>async function f(): Promise<T></code>	<code>async Task<T> F()</code>
<code>await p</code>	<code>await t</code>
<code>Promise.all([...])</code>	<code>Task.WhenAll(...)</code>
<code>Promise.race([...])</code>	<code>Task.WhenAny(...)</code>
<code>AbortController / signal</code>	<code>CancellationTokenSource / CancellationToken</code>
<code>for await (const x of stream)</code>	<code>await foreach (var x in stream)</code>

Khác biệt lớn: Node.js chạy trên **một luồng** với event loop. .NET có **thread pool** (nhiều luồng). `await` giải phóng luồng hiện tại trong lúc chờ I/O, để luồng đó phục vụ việc khác (ví dụ request HTTP khác). Đây là lý do ASP.NET Core chịu tải tốt.

Ví dụ

```
using System.Diagnostics;
using System.Runtime.CompilerServices;

var sw = Stopwatch.StartNew();

// 1. await tuần tự: tổng thời gian = cộng dồn
var user = await GetUserAsync(1);
var orders = await GetOrdersAsync(1);
Console.WriteLine($"Tuần tự: {user}, {orders} đơn - ~{Round(sw.ElapsedMilliseconds)} ms");

// 2. Chạy song song với Task.WhenAll: tổng thời gian = task lâu nhất
sw.Restart();
Task<string> userTask = GetUserAsync(2);
Task<int> ordersTask = GetOrdersAsync(2);
await Task.WhenAll(userTask, ordersTask);
Console.WriteLine($"WhenAll: {userTask.Result}, {ordersTask.Result} đơn - ~
{Round(sw.ElapsedMilliseconds)} ms");

// 3. WhenAll trả về mảng kết quả
```

```

int[] ids = [1, 2, 3];
string[] names = await Task.WhenAll(ids.Select(GetUserAsync));
Console.WriteLine($"Nhiều user: {string.Join(", ", names)}");

// 4. Exception trong async
try
{
    await GetUserAsync(-1);
}
catch (ArgumentException ex)
{
    Console.WriteLine($"Bắt lỗi async: {ex.Message}");
}

// 5. Cancellation với timeout
using var cts = new CancellationTokenSource(TimeSpan.FromMilliseconds(150));
try
{
    await SlowReportAsync(cts.Token);
}
catch (OperationCanceledException)
{
    Console.WriteLine("Báo cáo bị hủy sau 150 ms (timeout)");
}

// 6. IEnumerable + await foreach: stream dữ liệu
await foreach (var price in StreamPricesAsync(3))
    Console.WriteLine($" giá mới: {price}");

// 7. Task.WhenAny: lấy kết quả đầu tiên
var fast = Task.Delay(50).ContinueWith(_ => "server A");
var slow = Task.Delay(300).ContinueWith(_ => "server B");
var winner = await Task.WhenAny(fast, slow);
Console.WriteLine($"Nhanh nhất: {await winner}");

// 8. ValueTask cho đường đi nhanh có cache
var cache = new PriceCache();
Console.WriteLine($"Lần 1: {await cache.GetAsync("AAPL")} (từ nguồn), lần 2: {await
cache.GetAsync("AAPL")} (cache), số lần gọi nguồn = {cache.SourceCalls}");

static long Round(long ms) => (ms + 50) / 100 * 100; // làm tròn 100 ms để output ổn định

static async Task<string> GetUserAsync(int id)
{
    if (id < 0) throw new ArgumentException($"id không hợp lệ: {id}");
    await Task.Delay(200); // giả lập gọi DB
    return $"User{id}";
}

static async Task<int> GetOrdersAsync(int userId)
{
    await Task.Delay(300); // giả lập gọi API khác
    return userId * 2;
}

static async Task SlowReportAsync(CancellationToken ct)
{
    for (int i = 0; i < 10; i++)
    {
        await Task.Delay(100, ct); // ném OperationCanceledException khi token bị hủy
    }
}

static async IEnumerable<decimal> StreamPricesAsync(int count, [EnumeratorCancellation]
CancellationToken ct = default)
{
    decimal price = 100m;

```

```

    for (int i = 0; i < count; i++)
    {
        await Task.Delay(20, ct);
        price += 1.5m;
        yield return price;
    }
}

class PriceCache
{
    private readonly Dictionary<string, decimal> _cache = [];
    public int SourceCalls { get; private set; }

    public ValueTask<decimal> GetAsync(string symbol)
    {
        if (_cache.TryGetValue(symbol, out var cached))
            return ValueTask.FromResult(cached); // không cấp phát Task
        return new ValueTask<decimal>(LoadAsync(symbol));
    }

    private async Task<decimal> LoadAsync(string symbol)
    {
        SourceCalls++;
        await Task.Delay(10);
        return _cache[symbol] = 123.45m;
    }
}

```

Output thật (thời gian được làm tròn 100 ms để output ổn định giữa các lần chạy):

```

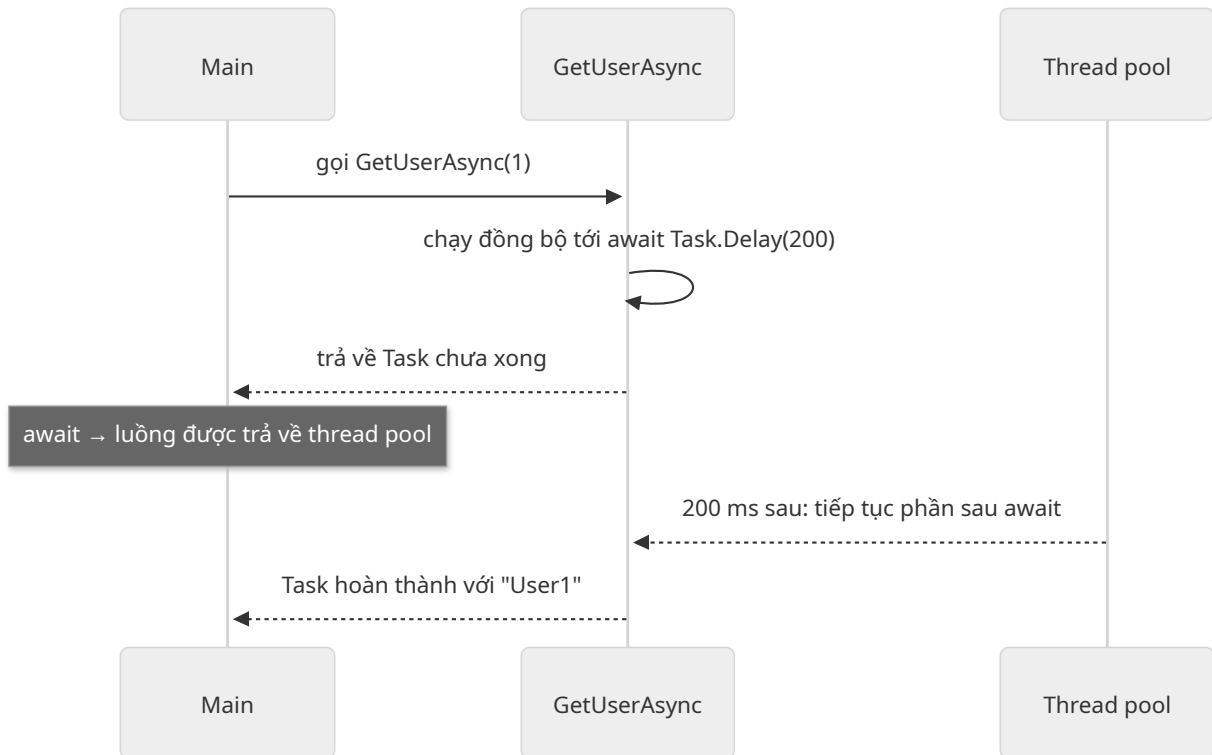
Tuần tự: User1, 2 đơn – ~500 ms
WhenAll: User2, 4 đơn – ~300 ms
Nhiều user: User1, User2, User3
Bắt lỗi async: id không hợp lệ: -1
Báo cáo bị hủy sau 150 ms (timeout)
    giá mới: 101.5
    giá mới: 103.0
    giá mới: 104.5
Nhanh nhất: server A
Lần 1: 123.45 (từ nguồn), lần 2: 123.45 (cache), số lần gọi nguồn = 1

```

- **Tuần tự** ~500 ms = 200 + 300. **WhenAll** ~300 ms = thời gian của tác vụ lâu nhất.
- Báo cáo 10 × 100 ms bị hủy ở 150 ms nhờ `CancellationTokenSource(TimeSpan)`.

Đi sâu

Chuyện gì xảy ra ở `await`?



Compiler biến hàm `async` thành một *state machine* (máy trạng thái). Mỗi `await` là một điểm dừng; phần code sau `await` được chạy tiếp khi tác vụ xong.

Bắt đầu task trước, `await` sau

`Task<string> userTask = GetUserAsync(2);` bắt đầu công việc ngay. Gọi hai hàm trước rồi mới `await Task.WhenAll(...)` → hai việc chạy chồng lên nhau. Nếu viết `await` ngay sau mỗi lời gọi → chạy tuần tự.

Exception với `async`

- Exception trong hàm `async` được lưu trong `Task`, và ném lại khi bạn `await`.
- `Task.WhenAll` ném exception **đầu tiên**; các exception khác nằm trong `task.Exception.InnerExceptions`.

Cancellation

- Người gọi tạo `CancellationTokenSource`, truyền `cts.Token` xuống.
- Hàm nhận `CancellationToken ct` và truyền tiếp cho mọi API hỗ trợ (`Task.Delay(ms, ct)`, `HttpClient.GetAsync(url, ct)`, `EF Core ToListAsync(ct)`).
- Khi bị hủy → `OperationCanceledException` (hoặc lớp con `TaskCanceledException`).
- Trong ASP.NET Core, `HttpContext.RequestAborted` là token bị hủy khi client ngắt kết nối.

ValueTask<T>

`Task<T>` là class → mỗi lần trả về tạo một object. `ValueTask<T>` là struct: khi kết quả có sẵn (cache hit), không cấp phát gì. Chỉ dùng khi đo đạc thấy cần; và **chỉ await một ValueTask một lần**.

Async và luồng: Task.Run

`async` **không** tự tạo luồng mới. Code CPU nặng (tính toán) trong hàm `async` vẫn chặn luồng. Dùng `Task.Run(() => ...)` để đẩy việc CPU sang thread pool (Tập 3, Chương 4).

Lỗi và bẫy thường gặp

- `.Result` / `.Wait()`: chặn luồng; trong một số môi trường (UI, ASP.NET cũ) gây *deadlock*. Dùng `await` từ đầu đến cuối.
- `async void`: người gọi không `await` được, exception làm sập tiến trình. Chỉ dùng cho event handler. Còn lại dùng `async Task`.
- **Quên await**: compiler cảnh báo CS4014; task chạy "mò côi", lỗi bị nuốt.
- **Không truyền CancellationTokens** xuống dưới → không hủy được thật sự.
- Dùng `Task.Delay` để "đợi cho chắc": dùng cơ chế đồng bộ đúng (Tập 3, Chương 4).
- **await trong vòng lặp khi các việc độc lập** → chậm. Dùng `Task.WhenAll`, nhưng giới hạn số lượng đồng thời nếu gọi dịch vụ ngoài (xem Bài tập 1).

Tóm tắt

- `Task<T>` $\approx$ `Promise<T>`; `await` giải phóng luồng trong lúc chờ.
- Bắt đầu nhiều task rồi `Task.WhenAll` để chạy song song.
- Luôn nhận và truyền `CancellationToken` trong code I/O.
- `IEnumerableAsync<T>` + `await foreach` để stream dữ liệu.

Bài tập (có lời giải)

1. "Tải" 6 trang (mỗi trang 100 ms) nhưng tối đa 2 trang cùng lúc. In số trang đồng thời tối đa.
2. Viết `WithTimeout(work, timeoutMs)`: trả về kết quả, hoặc chuỗi `timeout` nếu quá giờ.
3. Viết `RetryAsync<T>` thử lại khi gặp `HttpRequestException`, chờ tăng dần (20, 40, 80 ms...).

▼ Lời giải

```
using System.Diagnostics;

// Bài 1: tải 6 "trang" nhưng tối đa 2 cùng lúc (SemaphoreSlim)
var gate = new SemaphoreSlim(2);
int running = 0, maxRunning = 0;
var sw = Stopwatch.StartNew();
var pages = await Task.WhenAll(Enumerable.Range(1, 6).Select(async i =>
{
    await gate.WaitAsync();
    try
    {
        int now = Interlocked.Increment(ref running);
        InterlockedMax(ref maxRunning, now);
        await Task.Delay(100);
    }
}));
```

```

        return $"page{i}";
    }
    finally
    {
        Interlocked.Decrement(ref running);
        gate.Release();
    }
    });
    Console.WriteLine($"Bài 1: {pages.Length} trang, tối đa đồng thời = {maxRunning}, ~
{(sw.ElapsedMilliseconds + 50) / 100 * 100} ms");

    // Bài 2: WithTimeout – hủy nếu quá thời gian
    Console.WriteLine($"Bài 2: nhanh -> {await WithTimeout(ct => WorkAsync(50, ct), 200)}");
    Console.WriteLine($"Bài 2: chậm -> {await WithTimeout(ct => WorkAsync(500, ct), 200)}");

    // Bài 3: RetryAsync với backoff
    int calls = 0;
    var value = await RetryAsync(async () =>
    {
        calls++;
        await Task.Delay(10);
        if (calls < 3) throw new HttpRequestException($"503 lần {calls}");
        return "OK";
    }, maxAttempts: 4);
    Console.WriteLine($"Bài 3: {value} sau {calls} lần gọi");

    static async Task<string> WorkAsync(int ms, CancellationToken ct)
    {
        await Task.Delay(ms, ct);
        return $"xong sau {ms} ms";
    }

    static async Task<string> WithTimeout(Func<CancellationToken, Task<string>> work, int
timeoutMs)
    {
        using var cts = new CancellationTokenSource(timeoutMs);
        try { return await work(cts.Token); }
        catch (OperationCanceledException) { return $"timeout ({timeoutMs} ms)"; }
    }

    static async Task<T> RetryAsync<T>(Func<Task<T>> action, int maxAttempts)
    {
        for (int attempt = 1; ; attempt++)
        {
            try { return await action(); }
            catch (HttpRequestException ex) when (attempt < maxAttempts)
            {
                var delay = TimeSpan.FromMilliseconds(20 * Math.Pow(2, attempt - 1));
                Console.WriteLine($"Bài 3: lỗi '{ex.Message}', chờ {delay.TotalMilliseconds}
ms");
                await Task.Delay(delay);
            }
        }
    }

    static void InterlockedMax(ref int target, int value)
    {
        int current;
        while (value > (current = Volatile.Read(ref target)))
            Interlocked.CompareExchange(ref target, value, current);
    }

```

Bài 1: 6 trang, tối đa đồng thời = 2, ~300 ms
 Bài 2: nhanh -> xong sau 50 ms
 Bài 2: chậm -> timeout (200 ms)
 Bài 3: lỗi '503 lần 1', chờ 20 ms

Bài 3: lỗi '503 lần 2', chờ 40 ms
Bài 3: OK sau 3 lần gọi

Bài 1: SemaphoreSlim(2) là "cổng" chỉ cho 2 tác vụ vào cùng lúc; Interlocked đếm an toàn giữa nhiều luồng. 6 trang / 2 = 3 đợt $\times$ 100 ms $\approx$ 300 ms.

Nguồn tham khảo (Sources)

- Asynchronous programming with async and await: <https://github.com/dotnet/docs/blob/main/docs/csharp/asynchronous-programming/index.md>
- Task asynchronous programming model: <https://github.com/dotnet/docs/blob/main/docs/csharp/asynchronous-programming/task-asynchronous-programming-model.md>
- Cancel async tasks after a period of time: <https://github.com/dotnet/docs/blob/main/docs/csharp/asynchronous-programming/cancel-async-tasks-after-a-period-of-time.md>
- Async streams tutorial: <https://github.com/dotnet/docs/blob/main/docs/csharp/asynchronous-programming/generate-consume-asynchronous-stream.md>
- Async return types (ValueTask): <https://github.com/dotnet/docs/blob/main/docs/csharp/asynchronous-programming/async-return-types.md>

Chương 6 — Unit test với xUnit

Mục tiêu

- Tạo project test xUnit v3 và chạy bằng `dotnet test` trên .NET 10.
- Viết test theo mẫu Arrange–Act–Assert với `[Fact]` và `[Theory]`.
- Test exception, dùng *test double* (fake) thay cho dependency thật.
- Dùng fixture (`IClassFixture<T>`) và `MemberData`.

Giải thích đơn giản

Unit test là code kiểm tra một phần nhỏ (một class, một method) của chương trình. So sánh với công cụ bạn biết:

Jest / Vitest (TS)	JUnit 5 (Java)	xUnit (C#)
<code>test('...', () => {})</code>	<code>@Test</code>	<code>[Fact]</code>
<code>test.each([...])</code>	<code>@ParameterizedTest</code>	<code>[Theory]</code> + <code>[InlineData]</code>
<code>expect(x).toBe(y)</code>	<code>assertEquals(y, x)</code>	<code>Assert.Equal(y, x)</code>
<code>expect(fn).toThrow()</code>	<code>assertThrows</code>	<code>Assert.Throws<T>(...)</code>
<code>beforeAll</code>	<code>@BeforeAll</code>	<code>IClassFixture<T></code>
<code>beforeEach</code>	<code>@BeforeEach</code>	constructor của class test

Trong xUnit, **mỗi test chạy trên một object mới** của class test. Vì vậy constructor đóng vai trò "setup" cho từng test.

Ví dụ

Code cần test (class library `V2Ch06.Calc`)

```
namespace V2Ch06.Calc;

public interface IDiscountPolicy
{
    decimal GetDiscountRate(string customerTier);
}

public sealed class DefaultDiscountPolicy : IDiscountPolicy
{
    public decimal GetDiscountRate(string customerTier) => customerTier switch
    {
        "gold" => 0.10m,
        "silver" => 0.05m,
        _ => 0m,
    };
}

public record CartItem(string Sku, decimal UnitPrice, int Quantity);

public class PriceCalculator(IDiscountPolicy discountPolicy)
{

```

```

public const decimal VatRate = 0.08m;

public decimal Subtotal(IEnumerable<CartItem> items)
{
    ArgumentNullException.ThrowIfNull(items);
    decimal sum = 0;
    foreach (var item in items)
    {
        if (item.Quantity <= 0)
            throw new ArgumentOutOfRangeException(nameof(items), $"Số lượng của {item.Sku}
phải > 0");
        sum += item.UnitPrice * item.Quantity;
    }
    return sum;
}

public decimal Total(IEnumerable<CartItem> items, string customerTier)
{
    var subtotal = Subtotal(items);
    var discounted = subtotal * (1 - discountPolicy.GetDiscountRate(customerTier));
    return Math.Round(discounted * (1 + VatRate), 0, MidpointRounding.AwayFromZero);
}

```

PriceCalculator nhận IDiscountPolicy qua constructor — nhờ vậy test có thể truyền vào một "chính sách giảm giá giả" để kiểm soát kết quả.

Project test

```

<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <IsPackable>>false</IsPackable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="xunit.v3" Version="4.0.1" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include=" ../V2Ch06.Calc/V2Ch06.Calc.csproj" />
  </ItemGroup>

  <ItemGroup>
    <Using Include="Xunit" />
  </ItemGroup>

</Project>

```

- xunit.v3 4.0.1 (checked 2026-09-28) đã gồm *Microsoft.Testing.Platform* (MTP) runner.
- OutputType = Exe: project test xUnit v3 là một chương trình chạy được.
- File global.json của bộ sách có "test": { "runner": "Microsoft.Testing.Platform" }. Thiếu dòng này, .NET 10 SDK báo lỗi "Testing with VSTest target is no longer supported by Microsoft.Testing.Platform on .NET 10 SDK and later" (đã gặp thật khi dựng môi trường).

Test

```

using V2Ch06.Calc;

namespace V2Ch06.Calc.Tests;

```

```

public class PriceCalculatorTests
{
    // Fake (test double): thay thế dependency thật để test độc lập
    private sealed class FixedDiscount(decimal rate) : IDiscountPolicy
    {
        public decimal GetDiscountRate(string customerTier) => rate;
    }

    private static readonly CartItem[] Cart =
    [
        new("KB", 500_000m, 1),
        new("MOUSE", 250_000m, 2),
    ];

    [Fact]
    public void Subtotal_SumsUnitPriceTimesQuantity()
    {
        // Arrange
        var calc = new PriceCalculator(new FixedDiscount(0));

        // Act
        var subtotal = calc.Subtotal(Cart);

        // Assert
        Assert.Equal(1_000_000m, subtotal);
    }

    [Fact]
    public void Subtotal_EmptyCart_IsZero()
    {
        var calc = new PriceCalculator(new FixedDiscount(0));
        Assert.Equal(0m, calc.Subtotal([]));
    }

    [Fact]
    public void Subtotal_ZeroQuantity_Throws()
    {
        var calc = new PriceCalculator(new FixedDiscount(0));
        var ex = Assert.Throws<ArgumentOutOfRangeException>(() => calc.Subtotal([new("X", 10m,
0)]));
        Assert.Contains("X", ex.Message);
    }

    [Theory]
    [InlineData("gold", 972_000)] // 1,000,000 * 0.90 * 1.08
    [InlineData("silver", 1_026_000)] // 1,000,000 * 0.95 * 1.08
    [InlineData("basic", 1_080_000)] // 1,000,000 * 1.08
    public void Total_AppliesTierDiscountAndVat(string tier, int expected)
    {
        var calc = new PriceCalculator(new DefaultDiscountPolicy());
        Assert.Equal(expected, calc.Total(Cart, tier));
    }

    [Fact]
    public void Total_UsesInjectedPolicy()
    {
        var calc = new PriceCalculator(new FixedDiscount(0.5m));
        Assert.Equal(540_000m, calc.Total(Cart, "anything"));
    }
}

// Fixture: dữ liệu dùng chung cho nhiều test (tạo 1 lần cho cả class)
public sealed class CatalogFixture
{
    public Dictionary<string, decimal> Prices { get; } = new() { ["KB"] = 500_000m, ["MOUSE"]
= 250_000m };
}

```

```

    }

    public class CatalogTests(CatalogFixture fixture) : IClassFixture<CatalogFixture>
    {
        [Fact]
        public void Catalog_HasKeyboard() => Assert.True(fixture.Prices.ContainsKey("KB"));

        [Fact]
        public void Catalog_PricesArePositive() => Assert.All(fixture.Prices.Values, p =>
            Assert.True(p > 0));
    }

```

Chạy `dotnet test --output Detailed` (output thật; đã lược các dòng `from <đường dẫn dll>` lặp lại):

```

Running tests from <repo>/books/csharp/vol2-
intermediate/examples/V2Ch06.Calc.Tests/bin/Debug/net10.0/V2Ch06.Calc.Tests.dll (net10.0|x64)
passed V2Ch06.Calc.Tests.CatalogTests.Catalog_PricesArePositive (27ms)
passed V2Ch06.Calc.Tests.PriceCalculatorTests.Total_AppliesTierDiscountAndVat(tier: "silver",
expected: 1026000) (27ms)
passed V2Ch06.Calc.Tests.PriceCalculatorTests.Total_AppliesTierDiscountAndVat(tier: "gold", expected:
972000) (0ms)
passed V2Ch06.Calc.Tests.PriceCalculatorTests.Total_AppliesTierDiscountAndVat(tier: "basic", expected:
1080000) (0ms)
passed V2Ch06.Calc.Tests.PriceCalculatorTests.Total_UsesInjectedPolicy (0ms)
passed V2Ch06.Calc.Tests.CatalogTests.Catalog_HasKeyboard (1ms)
passed V2Ch06.Calc.Tests.PriceCalculatorTests.Subtotal_SumsUnitPriceTimesQuantity (0ms)
passed V2Ch06.Calc.Tests.PriceCalculatorTests.Subtotal_ZeroQuantity_Throws (3ms)
passed V2Ch06.Calc.Tests.PriceCalculatorTests.Subtotal_EmptyCart_IsZero (0ms)
<repo>/books/csharp/vol2-
intermediate/examples/V2Ch06.Calc.Tests/bin/Debug/net10.0/V2Ch06.Calc.Tests.dll (net10.0|x64) passed
(992ms)

Test run summary: Passed!
total: 9
failed: 0
succeeded: 9
skipped: 0
duration: 1s 291ms

```

Chú ý: thứ tự test trong output **không** theo thứ tự trong file. xUnit không đảm bảo thứ tự — test tốt không phụ thuộc vào nhau.

Đi sâu

Arrange – Act – Assert

1. **Arrange**: chuẩn bị object và dữ liệu.
2. **Act**: gọi đúng một hành động cần test.
3. **Assert**: kiểm tra kết quả.

Đặt tên test theo mẫu `Method_Condition_ExpectedResult`, ví dụ `Subtotal_ZeroQuantity_Throws`. Khi test fail, tên đã nói lên vấn đề.

Test double

Loại	Ý nghĩa	Ví dụ
Fake	cài đặt đơn giản nhưng chạy được	<code>FixedDiscount(0.5m)</code>

Loại	Ý nghĩa	Ví dụ
Stub	trả giá trị cố định	
Mock	kiểm tra được "đã gọi method nào, mấy lần"	thư viện như NSubstitute, Moq

Bộ sách dùng fake viết tay để không phụ thuộc thư viện thêm. Với project lớn, thư viện mock giúp viết nhanh hơn.

Fixture

- **Constructor + IDisposable**: setup/cleanup cho **mỗi** test.
- **IClassFixture<T>**: một object **T** dùng chung cho **mọi test trong class** (ví dụ `CatalogFixture`).
Hợp với tài nguyên đắt: database test, `WebApplicationFactory` (Tập 3).
- **Collection fixture**: dùng chung giữa nhiều class.

Chạy test chọn lọc

```
dotnet test --filter-method "**Subtotal*" # MTP: lọc theo tên method
dotnet test --output Detailed           # in từng test
```

(Các tùy chọn lọc của MTP khác VSTest cũ `--filter`. Xem `dotnet test --help`.)

Lỗi và bẫy thường gặp

- **Test phụ thuộc thứ tự hoặc trạng thái chung** (static field) → lúc pass lúc fail.
- **Test code dùng `DateTime.Now`, `random`, mạng** → không ổn định. Inject `IClock` (hoặc `TimeProvider` có sẵn từ .NET 8) như Chương 7.
- **Chỉ test "happy path"**: luôn thêm test cho dữ liệu sai, rỗng, biên.
- **Bẫy môi trường (gặp thật khi viết lời giải)**: bộ sách bật `InvariantGlobalization` để output ổn định. Ở chế độ này, `string.Normalize(NormalizationForm.FormD)` **không tách dấu tiếng Việt**, nên hàm tạo slug trả về `xin-chào-c` thay vì `xin-chao-c` và 6 test fail. Tài liệu của `dotnet/runtime` ghi rõ: ở invariant mode, yêu cầu normalize "trả về chuỗi gốc không đổi". Cách sửa: tắt `InvariantGlobalization` cho project cần xử lý Unicode (cần thư viện ICU trên Linux). Test đã phát hiện lỗi này — đúng việc của test!
- **Quên cấu hình `test.runner` trong `global.json`** với .NET 10 SDK + xUnit v3 → lỗi ngay khi chạy.

Tóm tắt

- xUnit: `[Fact]` cho một trường hợp, `[Theory]` cho nhiều dữ liệu.
- Viết code nhận dependency qua constructor để dễ thay bằng fake khi test.
- .NET 10 + xUnit v3 dùng `Microsoft.Testing.Platform`, bật trong `global.json`.

Bài tập (có lời giải)

Viết hàm `Slug.From(title)` chuyển tiêu đề tiếng Việt thành slug URL ("`Đường đến .NET 10!`" → "`duong-den-net-10`"), và viết test:

1. [Theory] với 3 tiêu đề và kết quả mong đợi.
2. Tiêu đề rỗng hoặc toàn khoảng trắng phải ném `ArgumentException`.
3. Dùng `[MemberData]` kiểm tra output chỉ chứa ký tự ASCII với các chữ `Á Æ Ć Ħ Ħ, Ơ Ư Đ`.

▼ Lời giải

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <IsPackable>>false</IsPackable>
    <!-- Normalize(FormD) cần ICU; chế độ InvariantGlobalization không tách dấu tiếng Việt -->
    <InvariantGlobalization>>false</InvariantGlobalization>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="xunit.v3" Version="4.0.1" />
  </ItemGroup>

  <ItemGroup>
    <Using Include="Xunit" />
  </ItemGroup>

</Project>
```

```
using System.Globalization;
using System.Text;

namespace V2Ch06.Solutions.Tests;

// Code cần test: tạo "slug" cho URL từ tiêu đề tiếng Việt
public static class Slug
{
    public static string From(string title)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(title);
        var normalized = title.Replace('đ', 'd').Replace('Đ',
'D').Normalize(NormalizationForm.FormD);
        var sb = new StringBuilder();
        foreach (var c in normalized)
        {
            if (CharUnicodeInfo.GetUnicodeCategory(c) == UnicodeCategory.NonSpacingMark)
continue; // bỏ dấu
            if (char.IsLetterOrDigit(c)) sb.Append(char.ToLowerInvariant(c));
            else if (sb.Length > 0 && sb[^1] != '-') sb.Append('-');
        }
        return sb.ToString().Trim('-');
    }
}

public class SlugTests
{
    // Bài 1: Theory với nhiều dữ liệu
    [Theory]
    [InlineData("Xin chào C#", "xin-chao-c")]
    [InlineData("Đường đến .NET 10!", "duong-den-net-10")]
    [InlineData(" Nhiều khoảng trắng ", "nhieu-khoang-trang")]
    public void From_CreatesSlug(string title, string expected) => Assert.Equal(expected,
Slug.From(title));

    // Bài 2: test exception
    [Theory]
    [InlineData("")]
    [InlineData(" ")]
```

```

    public void From_Blank_Throws(string title) => Assert.Throws<ArgumentException>(() =>
Slug.From(title));

    // Bài 3: MemberData cho dữ liệu phức tạp hơn
    public static TheoryData<string> Titles => new() { "Ắ Ắ Ắ Ắ Ặ", "Ơ Ư Đ", "Lập trình" };

    [Theory]
    [MemberData(nameof(Titles))]
    public void From_OutputIsAscii(string title) => Assert.All(Slug.From(title), c =>
Assert.True(c < 128));
}

```

```

Running tests from <repo>/books/csharp/vol2-
intermediate/examples/V2Ch06.Solutions.Tests/bin/Debug/net10.0/V2Ch06.Solutions.Tests.dll
(net10.0|x64)
passed V2Ch06.Solutions.Tests.SlugTests.From_CreatesSlug(title: "Xin chào C#", expected: "xin-chao-
c") (17ms)
passed V2Ch06.Solutions.Tests.SlugTests.From_CreatesSlug(title: "  Nhiều  khoảng  trắng  ",
expected: "nhieu-khoang-trang") (0ms)
passed V2Ch06.Solutions.Tests.SlugTests.From_CreatesSlug(title: "Đường đến .NET 10!", expected:
"duong-den-net-10") (0ms)
passed V2Ch06.Solutions.Tests.SlugTests.From_Blank_Throws(title: "  ") (1ms)
passed V2Ch06.Solutions.Tests.SlugTests.From_Blank_Throws(title: "") (0ms)
passed V2Ch06.Solutions.Tests.SlugTests.From_OutputIsAscii(title: "Lập trình") (1ms)
passed V2Ch06.Solutions.Tests.SlugTests.From_OutputIsAscii(title: "Ắ Ắ Ắ Ắ Ặ") (0ms)
passed V2Ch06.Solutions.Tests.SlugTests.From_OutputIsAscii(title: "Ơ Ư Đ") (0ms)
<repo>/books/csharp/vol2-
intermediate/examples/V2Ch06.Solutions.Tests/bin/Debug/net10.0/V2Ch06.Solutions.Tests.dll
(net10.0|x64) passed (847ms)

Test run summary: Passed!
total: 8
failed: 0
succeeded: 8
skipped: 0
duration: 1s 140ms

```

đ/Đ không phải "chữ d + dấu" trong Unicode, nên phải thay thủ công trước khi `Normalize`.

Nguồn tham khảo (Sources)

- Unit testing C# with xUnit (nguồn Microsoft Learn): <https://github.com/dotnet/docs/blob/main/docs/core/testing/unit-testing-csharp-with-xunit.md>
- Testing with `dotnet test` (MTP, global.json): <https://github.com/dotnet/docs/blob/main/docs/core/testing/unit-testing-with-dotnet-test.md>
- xUnit.net repository: <https://github.com/xunit/xunit>
- xunit.v3 package versions (NuGet API): <https://api.nuget.org/v3-flatcontainer/xunit.v3/index.json>
- Globalization invariant mode: <https://github.com/dotnet/runtime/blob/main/docs/design/features/globalization-invariant-mode.md>

Chương 7 — Dependency injection

Mục tiêu

- Hiểu *dependency injection* (DI — tiêm phụ thuộc) và vì sao nó giúp code dễ test.
- Đăng ký service với 3 *lifetime*: Singleton, Scoped, Transient.
- Dùng *Generic Host* (`Host.CreateApplicationBuilder()`) với configuration, options và logging.
- Dùng nhiều implementation, *keyed services* và mẫu decorator.

Giải thích đơn giản

Bạn là Angular developer — bạn **đã dùng DI mỗi ngày**:

```
@Injectable({ providedIn: 'root' })
export class OrderService {
  constructor(private http: HttpClient, private logger: Logger) {}
}
```

(Đoạn Angular trên chỉ để minh họa, không chạy trong bộ sách.)

Angular tạo `HttpClient` và `Logger` rồi "tiêm" vào constructor. .NET làm y hệt:

Angular	.NET (<code>Microsoft.Extensions.DependencyInjection</code>)
<code>@Injectable({ providedIn: 'root' })</code>	<code>services.AddSingleton<IClock, SystemClock>()</code>
provider ở component (mỗi component một instance)	<code>AddScoped</code> (mỗi HTTP request một instance)
<code>useFactory</code>	<code>services.AddSingleton<IFoo>(sp => new Foo(...))</code>
<code>InjectionToken</code>	keyed service <code>AddKeyedSingleton<T, TImpl>("key")</code>
<code>inject(Foo)</code>	<code>provider.GetRequiredService<Foo>()</code>

Spring (Java) cũng vậy: `@Service` + constructor injection.

Ví dụ

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.Hosting" Version="10.0.12" />
  </ItemGroup>
</Project>
```

```
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using Microsoft.Extensions.Options;
```

```

// 1. Container DI "trần": ServiceCollection
var services = new ServiceCollection();
services.AddSingleton<IClock, FixedClock>(); // 1 instance cho cả ứng dụng
services.AddScoped<RequestContext>(); // 1 instance mỗi scope (mỗi HTTP
request)
services.AddTransient<IGreeter, VietnameseGreeter>(); // instance mới mỗi lần xin

using (var provider = services.BuildServiceProvider(validateScopes: true))
{
    for (int request = 1; request <= 2; request++)
    {
        using var scope = provider.CreateScope();
        var sp = scope.ServiceProvider;
        var ctx1 = sp.GetRequiredService<RequestContext>();
        var ctx2 = sp.GetRequiredService<RequestContext>();
        var g1 = sp.GetRequiredService<IGreeter>();
        var g2 = sp.GetRequiredService<IGreeter>();
        bool sameClock = ReferenceEquals(sp.GetRequiredService<IClock>(),
provider.GetRequiredService<IClock>());
        Console.WriteLine($"Request {request}: scoped cùng object? {ReferenceEquals(ctx1,
ctx2)} (id={ctx1}), " +
            $"transient cùng object? {ReferenceEquals(g1, g2)}, singleton cùng
object? {sameClock}");
        Console.WriteLine(" " + g1.Greet("Nobin"));
    }
}

// 2. Generic Host: DI + configuration + logging + options (giống ASP.NET Core)
var builder = Host.CreateApplicationBuilder(args);
builder.Logging.ClearProviders();
builder.Logging.AddSimpleConsole(o => { o.SingleLine = true; o.IncludeScopes = false; });
builder.Configuration.AddInMemoryCollection(new Dictionary<string, string?>
{
    ["Email:Sender"] = "noreply@shop.vn",
    ["Email:MaxRetries"] = "3",
});
builder.Services.Configure<EmailOptions>(builder.Configuration.GetSection("Email"));
builder.Services.AddSingleton<IClock, FixedClock>();
builder.Services.AddTransient<IEmailSender, FakeEmailSender>();
builder.Services.AddTransient<OrderService>();

using var host = builder.Build();
var orderService = host.Services.GetRequiredService<OrderService>();
orderService.PlaceOrder("an@example.com", 250_000m);

// 3. Lỗi thường gặp: quên đăng ký service
try
{
    host.Services.GetRequiredService<IGreeter>();
}
catch (InvalidOperationException ex)
{
    Console.WriteLine($"Lỗi DI: {ex.Message}");
}

public interface IClock { DateTime Now { get; } }
public sealed class FixedClock : IClock { public DateTime Now => new(2026, 9, 28, 9, 0, 0); }

public sealed class RequestContext { public Guid Id { get; } = Guid.NewGuid(); public override
string ToString() => Id.ToString()[..8]; }

public interface IGreeter { string Greet(string name); }
public sealed class VietnameseGreeter(IClock clock) : IGreeter
{
    public string Greet(string name) => $"Chào buổi {(clock.Now.Hour < 12 ? "sáng" :
"chiều")}, {name}!";
}

```

```

    }

    public sealed class EmailOptions
    {
        public string Sender { get; set; } = "";
        public int MaxRetries { get; set; }
    }

    public interface IEmailSender { void Send(string to, string subject); }

    public sealed class FakeEmailSender(IOptions<EmailOptions> options, ILogger<FakeEmailSender>
logger) : IEmailSender
    {
        public void Send(string to, string subject) =>
            logger.LogInformation("Gửi email từ {Sender} tới {To}: {Subject} (retry tối đa
{MaxRetries})",
                options.Value.Sender, to, subject, options.Value.MaxRetries);
    }

    // Constructor injection: OrderService không tự tạo dependency
    public sealed class OrderService(IEmailSender email, IClock clock, ILogger<OrderService>
logger)
    {
        public void PlaceOrder(string customerEmail, decimal amount)
        {
            logger.LogInformation("Đặt hàng {Amount} lúc {Time:HH:mm}", amount, clock.Now);
            email.Send(customerEmail, $"Xác nhận đơn hàng {amount:N0} VND");
        }
    }
}

```

Output thật (id của scope là Guid ngẫu nhiên, mỗi lần chạy khác nhau):

```

Request 1: scoped cùng object? True (id=ad6fbab4), transient cùng object? False, singleton cùng
object? True
    Chào buổi sáng, Nabin!
Request 2: scoped cùng object? True (id=5ccfe51f), transient cùng object? False, singleton cùng
object? True
    Chào buổi sáng, Nabin!
info: OrderService[0] Đặt hàng 250000 lúc 09:00
info: FakeEmailSender[0] Gửi email từ noreply@shop.vn tới an@example.com: Xác nhận đơn hàng 250,000
VND (retry tối đa 3)
Lỗi DI: No service for type 'IGreeter' has been registered.

```

Đi sâu

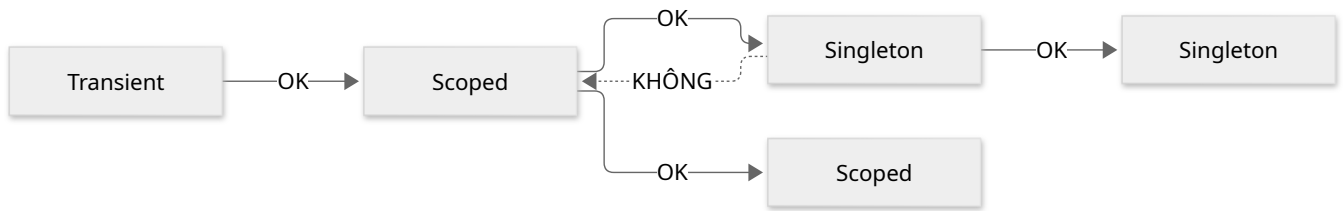
Ba lifetime

Lifetime	Tạo instance	Ví dụ dùng
Singleton	1 lần cho cả ứng dụng	cache, cấu hình, HttpClient factory, clock
Scoped	1 lần mỗi scope (ASP.NET Core: mỗi request)	DbContext của EF Core, thông tin user hiện tại
Transient	mỗi lần được xin	service nhẹ, không trạng thái

Output chứng minh: trong cùng request, scoped là **cùng object** (True), transient là object **khác** (False); hai request có id scoped khác nhau.

Quy tắc "captive dependency"

Service sống lâu **không được** giữ service sống ngắn hơn: Singleton không được nhận Scoped qua constructor (DbContext sẽ bị dùng chung giữa các request → lỗi). `validateScopes: true` (bật sẵn trong môi trường Development của ASP.NET Core) sẽ ném exception khi phát hiện.



Generic Host

`Host.CreateApplicationBuilder(args)` tạo sẵn:

- **Configuration:** đọc `appsettings.json`, biến môi trường, tham số dòng lệnh (ví dụ dùng `AddInMemoryCollection` để tự chứa).
- **Logging:** `ILogger<T>` inject được ở mọi nơi.
- **Options:** `services.Configure<EmailOptions>(section)` rồi inject `IOptions<EmailOptions>`.
- **DI container:** `builder.Services`.

ASP.NET Core (`WebApplication.CreateBuilder`) dùng đúng mô hình này — Tập 3 sẽ thấy lại.

Structured logging

`logger.LogInformation("Gửi email từ {Sender} tới {To}", sender, to)` — **không** dùng `$"..."`. `{Sender}` là tên trường; hệ thống log lưu cả giá trị riêng để tìm kiếm/lọc (Tập 3, Chương 6).

DI giúp test

`OrderService` chỉ biết `ISender`, `IClock`. Trong test, bạn truyền `FakeEmailSender` và `FixedClock` — không gửi email thật, thời gian cố định. Đây chính là kỹ thuật ở Chương 6.

Lỗi và bẫy thường gặp

- **Quên đăng ký service** → `InvalidOperationException: No service for type ... has been registered` (xem output).
- **Captive dependency** (Singleton giữ Scoped) → dữ liệu lẫn giữa request, lỗi đa luồng.
- **Service locator:** gọi `provider.GetService<T>()` rải rác trong code thay vì constructor injection → khó đọc, khó test. Chỉ dùng ở "gốc" ứng dụng.
- **Tự new dependency** bên trong class → mất khả năng thay thế khi test.
- **Dispose:** container tự `Dispose` service nó tạo khi scope/provider kết thúc. Đừng tự `Dispose` service được inject.

Tóm tắt

- DI trong .NET giống DI trong Angular/Spring: đăng ký rồi inject qua constructor.

- Singleton / Scoped / Transient quyết định vòng đời object.
- Generic Host gom DI + configuration + logging + options — nền tảng của ASP.NET Core.

Bài tập (có lời giải)

1. Đăng ký 2 `INotifier` (email, sms) và cho `AlertService` nhận `IEnumerable<INotifier>` để gửi qua tất cả.
2. Dùng keyed services để chọn cổng thanh toán "momo" hoặc "vnpay".
3. Viết decorator `LoggingProductRepository` bọc `SqlProductRepository` và đăng ký sao cho người dùng `IProductRepository` nhận bản có log.

▼ Lời giải

```
using Microsoft.Extensions.DependencyInjection;

var services = new ServiceCollection();

// Bài 1: nhiều implementation cho cùng interface -> inject IEnumerable<T>
services.AddSingleton<INotifier, EmailNotifier>();
services.AddSingleton<INotifier, SmsNotifier>();
services.AddSingleton<AlertService>();

// Bài 2: keyed services (.NET 8+): chọn implementation theo khóa
services.AddKeyedSingleton<IPaymentGateway, MomoGateway>("momo");
services.AddKeyedSingleton<IPaymentGateway, VnPayGateway>("vnpay");

// Bài 3: decorator thủ công: thêm log quanh repository thật
services.AddSingleton<SqlProductRepository>();
services.AddSingleton<IProductRepository>(sp => new
LoggingProductRepository(sp.GetRequiredService<SqlProductRepository>()));

using var provider = services.BuildServiceProvider();

provider.GetRequiredService<AlertService>().Alert("Server CPU 95%");

foreach (var key in new[] { "momo", "vnpay" })
    Console.WriteLine($"Bài 2: {provider.GetRequiredKeyedService<IPaymentGateway>
(key).Pay(150_000m)}");

var repo = provider.GetRequiredService<IProductRepository>();
Console.WriteLine($"Bài 3: kết quả = {repo.GetName(7)}");

public interface INotifier { string Channel { get; } void Send(string message); }
public sealed class EmailNotifier : INotifier { public string Channel => "email"; public
void Send(string m) => Console.WriteLine($"Bài 1: [email] {m}"); }
public sealed class SmsNotifier : INotifier { public string Channel => "sms"; public void
Send(string m) => Console.WriteLine($"Bài 1: [sms] {m}"); }

public sealed class AlertService(IEnumerable<INotifier> notifiers)
{
    public void Alert(string message)
    {
        foreach (var n in notifiers) n.Send(message);
    }
}

public interface IPaymentGateway { string Pay(decimal amount); }
public sealed class MomoGateway : IPaymentGateway { public string Pay(decimal a) => $"MoMo
thanh toán {a:N0}"; }
public sealed class VnPayGateway : IPaymentGateway { public string Pay(decimal a) =>
$"VNPay thanh toán {a:N0}"; }
```

```

public interface IProductRepository { string GetName(int id); }
public sealed class SqlProductRepository : IProductRepository { public string GetName(int
id) => $"Sản phẩm #{id}"; }
public sealed class LoggingProductRepository(IProductRepository inner) : IProductRepository
{
    public string GetName(int id)
    {
        Console.WriteLine($"Bài 3: [log] GetName({id}) bắt đầu");
        var result = inner.GetName(id);
        Console.WriteLine($"Bài 3: [log] GetName xong");
        return result;
    }
}

```

```

Bài 1: [email] Server CPU 95%
Bài 1: [sms] Server CPU 95%
Bài 2: MoMo thanh toán 150,000
Bài 2: VNPAY thanh toán 150,000
Bài 3: [log] GetName(7) bắt đầu
Bài 3: [log] GetName xong
Bài 3: kết quả = Sản phẩm #7

```

Bài 3 dùng *factory registration* `AddSingleton<IProductRepository>(sp => ...)` để tự động chuỗi decorator.

Nguồn tham khảo (Sources)

- Dependency injection in .NET: <https://github.com/dotnet/docs/blob/main/docs/core/extensions/dependency-injection/overview.md>
- Service lifetimes: <https://github.com/dotnet/docs/blob/main/docs/core/extensions/dependency-injection/service-lifetimes.md>
- DI guidelines (captive dependency): <https://github.com/dotnet/docs/blob/main/docs/core/extensions/dependency-injection/guidelines.md>
- .NET Generic Host: <https://github.com/dotnet/docs/blob/main/docs/core/extensions/generic-host.md>
- Options pattern: <https://github.com/dotnet/docs/blob/main/docs/core/extensions/options.md>
- Microsoft.Extensions.Hosting on NuGet: <https://www.nuget.org/packages/Microsoft.Extensions.Hosting/10.0.12>

GLOSSARY — Thuật ngữ (Tiếng Anh → Tiếng Việt)

Sắp xếp theo thứ tự chữ cái. Cột "Tập" cho biết thuật ngữ xuất hiện lần đầu ở đâu.

Thuật ngữ	Giải thích tiếng Việt	Tập
abstract class	class trừu tượng, không tạo object trực tiếp, dùng làm lớp cha	1
access modifier	từ khóa phạm vi truy cập: <code>public</code> , <code>private</code> , <code>protected</code> , <code>internal</code>	1
ArrayPool	kho mảng dùng lại, giảm cấp phát bộ nhớ	3
assembly	file <code>.dll</code> / <code>.exe</code> kết quả build một project	1
async / await	cú pháp viết code bất đồng bộ như code tuần tự	1, 2
back-pressure	cơ chế làm bên gửi chậm lại khi bên nhận xử lý không kịp	3
backing field	field ẩn chứa giá trị thật của property	1
BenchmarkDotNet	thư viện đo hiệu năng chuẩn của .NET	3
boxing	đóng gói value type vào object trên heap (tốn bộ nhớ)	2
Channel<T>	hàng đợi bất đồng bộ cho mẫu producer/consumer	3
checked / unchecked	bật/tắt kiểm tra tràn số	1
clean architecture	kiến trúc chia tầng, phụ thuộc chỉ hướng vào trong	3
closure	lambda "nhớ" biến ở phạm vi bên ngoài	2
collection expression	cú pháp <code>[1, 2, 3]</code> tạo collection; <code>..</code> là spread	1
collection	tập hợp nhiều phần tử: <code>List<T></code> , <code>Dictionary</code> , ...	1
constraint (generic)	ràng buộc <code>where T : ...</code> cho tham số kiểu	2
constructor	hàm khởi tạo object	1
container / image (Docker)	image: gói ứng dụng + môi trường; container: image đang chạy	3
DbContext	phiên làm việc với database trong EF Core	3
deadlock	hai luồng chờ nhau mãi mãi	3
decimal	kiểu số thập phân 128 bit, dùng cho tiền	1
deconstruction	tách một giá trị thành nhiều biến: <code>var (a, b) = tuple;</code>	1
deferred execution	thực thi trì hoãn: query LINQ chỉ chạy khi duyệt	2
delegate	kiểu đại diện cho một hàm (con trỏ hàm an toàn kiểu)	2
dependency injection (DI)	tiêm phụ thuộc: object nhận dependency từ bên ngoài thay vì tự tạo	2
Dispose / IDisposable	giải phóng tài nguyên (file, kết nối)	1
DTO (Data Transfer Object)	object chỉ chở dữ liệu request/response	2, 3
EF Core	Entity Framework Core — ORM của .NET	3
endpoint filter	bộ lọc chạy quanh một endpoint Minimal API	3
event	cơ chế thông báo dựa trên delegate	2
exception filter	<code>catch (X) when (...)</code> : chỉ bắt khi điều kiện đúng	1
exception	ngoại lệ, object mô tả lỗi lúc chạy	1

Thuật ngữ	Giải thích tiếng Việt	Tập
extension member	thêm method/property cho kiểu có sẵn mà không sửa kiểu đó (C# 14 mở rộng)	1, 2
field keyword	từ khóa C# 14 truy cập backing field trong property	1
file-based app	ứng dụng chỉ một file <code>.cs</code> , chạy bằng <code>dotnet run app.cs</code> (.NET 10)	1
fixture (test)	dữ liệu/tài nguyên dùng chung giữa các test	2
GC (Garbage Collector)	bộ gom rác, tự giải phóng object không còn dùng	3
Generic Host	"khung" ứng dụng gồm DI, configuration, logging	2
generic	kiểu/hàm có tham số kiểu <code><T></code>	1, 2
health check	endpoint báo ứng dụng còn sống / sẵn sàng	3
heap / stack	vùng nhớ cho object (heap) và cho biến cục bộ, lời gọi hàm (stack)	1
IL (Intermediate Language)	ngôn ngữ trung gian mà C# biên dịch ra	1
immutable	bất biến, không sửa được sau khi tạo	1
init accessor	<code>init</code> : chỉ gán lúc khởi tạo object	1
integration test	test nhiều thành phần cùng chạy thật (HTTP, DB)	3
interface	"hợp đồng" gồm các thành viên mà class phải cài đặt	1
Interlocked	thao tác nguyên tử trên biến số, an toàn đa luồng	3
JIT (Just-In-Time)	biên dịch IL sang mã máy lúc chạy	1
lifetime (DI)	vòng đời service: Singleton, Scoped, Transient	2
LINQ	Language Integrated Query: truy vấn dữ liệu bằng <code>Where</code> , <code>Select</code> ...	1, 2
LTS / STS	Long/Standard Term Support: hỗ trợ dài hạn (3 năm) / ngắn hạn	1
Microsoft.Testing.Platform (MTP)	nền tảng chạy test mới, dùng với xUnit v3 trên .NET 10	2
middleware	thành phần xử lý request/response theo chuỗi	3
migration (EF Core)	file mô tả thay đổi schema database theo thời gian	3
Minimal API	cách viết Web API bằng lambda <code>MapGet/MapPost...</code>	3
namespace	không gian tên, nhóm các kiểu	1
NuGet	kho và trình quản lý gói của .NET	1
null-coalescing	toán tử <code>??</code> và <code>??=</code>	1
null-conditional	toán tử <code>?.</code> và <code>?[]</code>	1
nullable	có thể mang giá trị <code>null</code> : <code>int?</code> , <code>string?</code>	1
observability	khả năng quan sát hệ thống qua logs, traces, metrics	3
OpenTelemetry	chuẩn mở thu thập và xuất logs/traces/metrics	3
ORM	ánh xạ object ↔ bảng database	3
override / virtual	ghi đè method của lớp cha / cho phép ghi đè	1
pattern matching	so khớp mẫu: <code>is</code> , <code>switch</code> với pattern	1, 2
primary constructor	tham số constructor khai báo ngay sau tên class	1
Problem Details	định dạng JSON chuẩn cho lỗi HTTP (RFC 7807, bản mới RFC 9457)	3

Thuật ngữ	Giải thích tiếng Việt	Tập
property	thuộc tính có <code>get / set</code>	1
race condition	kết quả sai do nhiều luồng cùng sửa dữ liệu	3
record	kiểu dữ liệu so sánh theo giá trị, hỗ trợ <code>with</code>	1, 2
reference type	kiểu tham chiếu: gán là copy địa chỉ	1
reified generics	generic giữ thông tin kiểu lúc chạy (khác type erasure của Java)	2
required member	thành viên bắt buộc gán khi khởi tạo	1
runtime	môi trường chạy chương trình (.NET Runtime / CLR)	1
scope (logging)	ngữ cảnh gắn thêm trường cho mọi log bên trong	3
SDK	bộ công cụ phát triển (<code>dotnet CLI, compiler</code>)	1
sealed	không cho kế thừa tiếp	1
<code>Span<T></code>	"cửa sổ" nhìn vào vùng nhớ liên tục, không copy	3
<code>stackalloc</code>	cấp phát bộ nhớ tạm trên stack	3
<code>struct</code>	kiểu giá trị do người dùng định nghĩa	1
structured logging	log có cấu trúc: giữ tên và giá trị từng trường	2, 3
switch expression	<code>x switch { pattern => value, ... }</code> trả về giá trị	1
test double / fake / mock	đối tượng giả thay dependency thật khi test	2
Theory / Fact (xUnit)	test có dữ liệu tham số / test đơn	2
top-level statements	viết code trực tiếp trong <code>Program.cs</code> , không cần <code>Main</code>	1
trace / span	hành trình của một request / một bước trong hành trình đó	3
tuple	nhóm nhiều giá trị: <code>(int Min, int Max)</code>	1
use case	trường hợp sử dụng: một thao tác nghiệp vụ của ứng dụng	3
value type	kiểu giá trị: gán là copy dữ liệu	1
variance (in/out)	quy tắc gán generic giữa kiểu cha/con	2